Software Requirements Specification (SRS)

IoT-Enabled Smart bus route monitoring and revenue management system

Project ID

R26-ISE-004

Student ID	Student Name
IT22228208	Gunasekara W S H R
IT22121042	Ahamed M T M
IT22264152	Sachintha H D D
IT22925268	Aththanayaka N D

Supervisor: Mr. Sanjeewa Perera

Date of Submission: 13/05/2026

B.Sc. (Hons) Degree in Information Technology Specialized in Information System Engineering

Sri Lanka Institute of Information Technology Sri Lanka

May 2026

Table of Contents

Contents

LIST OF FIGURES 4

LIST OF APPENDICES 5

1. INTRODUCTION 6

1.1 Purpose 6

1.2 Scope 7

1.3 Definitions, Acronyms, & Abbreviations 8

2. Overall Description 10

2.1 Product Perspective 11

3. Specific Requirement (DD) 33

3.1 External Interface Requirements 33

3.2.1 High-Level Architectural Design 40

3.2.2 Hardware and Software requirements with justification 44

4. Supporting Information 62

4.1APPENDICES 62

LIST OF TABLES

Table 1 User Interface 16
Table 2 Hardware Interfaces 17
Table 3 Software Interfaces and purpose 19
Table 4 Communication Interfaces 20
Table 5 Memory Constraints 21
Table 6 Normal Operations 22
Table 7 Special Operations 23
Table 8 User Characteristics 28
Table 9 Apportioning of Requirements 32
Table 10 User Interfaces Requirements 34
Table 11 Hardware Requirements 36
Table 12 Software Requirements 38
Table 13 Communication Interfaces 40
Table 14 Component Interaction 43
Table 15 Hardware Requirements 45
Table 16 Software Requirements 46
Table 17 Risk Mitigation Plan 48
Table 18 Budget 49
Table 19 Expected Benefits 50
Table 20 Performance Requirements 52
Table 21 Availability Requirements 56
Table 22 Security 57
Table 23 Maintainability 59

LIST OF FIGURES

Figure 1 High-Level Architecture Diagram 42
Figure 2 High-Level Architecture Flow..... 43
Figure 3 UI/ UX Route Assignment..... 62
Figure 4 UI/ UX Route..... 63
Figure 5 VS Code Sechedule 63
Figure 6 VS Code Tracking Backend..... 64
Figure 7 UI/ UX App 1 65
Figure 8 UI/ UX App 2 66
Figure 9 Revenue Calculatin 67
Figure 10 UI/ UX Tracking 67
Figure 11 Device Making step 1 68
Figure 12 Device Making step 2 68
Figure 13 Final Device 69
Figure 14 Current use bus schedule 70

LIST OF APPENDICES

Appendix	Title	Page
Appendix A	Survey / Interview Instruments	63
Appendix B	Detailed System Architecture Diagram	65
Appendix C	Preliminary UI Mockups	67
Appendix D	API Specifications	70
Appendix E	Ethical Considerations	72

1. INTRODUCTION

1.1 Purpose

The Software Requirements Specification (SRS) document establishes all functional and non-functional requirements needed for the IoT-Enabled Smart Bus Route Monitoring and Revenue Management System. The document establishes a complete requirement foundation which the system development team will use for system development and integration and testing and validation and stakeholder communication and future maintenance work. The document defines system functions together with the user roles who will access the system and the system interfaces and performance requirements and implementation requirements that developers must follow.

The research project team and their supervisor and evaluators and transport authority stakeholders and developers and testers and future system maintainers will use this document.

1.2 Scope

The new system functions as an intelligent bus management system, which manages Sri Lankan public bus services, especially those operated by private bus companies that the transport authority supervises. The system uses IoT technology to collect bus events while it monitors route compliance, and it uses policy-based scheduling with time-slot classification and distance-based systems to calculate revenue, driver monitoring, and operational reporting functions. The SRS document defines all system components, which include software platform elements, backend API specifications, database requirements, web dashboard design, mobile application development, IoT event processing system, and system integration needs. The document presents embedded device design information at the interface level because it follows the SRS structure instead of embedded system design description standards.

In Scope: In-scope functions include:

- User authentication, authorization, and role-based access control.
- Bus, route, driver, owner, timetable, policy, and time-slot management.
- IoT event receiving from onboard GPS/sensor devices.
- Event-driven bus monitoring, halt detection, trip start/end detection, and route deviation detection.
- Policy-driven time-slot classification into Peak, Normal, Night/Off-Peak, or configured authority-defined periods.
- Automated schedule adjustment support during delays, breakdowns, unavailable buses, or service gaps.
- Distance-based permit fee calculation and off-peak incentive/commission calculation.
- Driver behavior monitoring and unsafe driving event alerts using speed, acceleration, and gyroscope data.
- Authority web dashboard and driver/owner mobile application.
- Operational, compliance, revenue, incentive, and management reporting.

Out of Scope: Out-of-scope items include:

- Final production-grade hardware enclosure manufacturing.
- Integration with real national payment gateways, unless approved as a later extension.
- Public passenger-facing ETA application unless added as a future enhancement.
- Legal enforcement decisions themselves; the system provides evidence, alerts, and reports for authorized officers.

- Full island-wide deployment during the research prototype phase.

1.3 Definitions, Acronyms, & Abbreviations

Term	Definition
API	Application Programming Interface used for communication between frontend, backend, mobile app, and services.
DSR	Design Science Research methodology is used to design, build, and evaluate the system artifact.
ESP32	Microcontroller used in the onboard IoT device for processing and communication.
GPS	The Global Positioning System is used for collecting bus location data.
GIS	A Geographic Information System is used for route maps and spatial analysis.
IoT	Internet of Things: connected physical devices that collect and transmit data.
IMU	Inertial Measurement Unit; sensor data from accelerometer and gyroscope.
MPU6050	A six-axis accelerometer and gyroscope sensor used for driver behavior detection.
Off-Peak/Night Slot	Low-demand service period used for incentive calculation and service availability monitoring.
Policy Rule	A configurable rule defined by a transport authority for scheduling, time-slot classification, revenue, or compliance.
RPTA	Road Passenger Transport Authority.
Route Deviation	Movement outside the permitted or approved route beyond a configured threshold.
SRS	Software Requirements Specification.
Time Slot	Predefined service period, such as Peak, Normal, Night, or authority-configured service category.

1.4 Overview

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System is a software-based transport management platform integrated with IoT devices to improve the reliability, transparency, and efficiency of public bus operations in Sri Lanka. The system tracks bus location and trip progress and route compliance and driver behavior and operational events through its use of GPS and sensor data from installed onboard IoT devices. The data streams through a centralized backend system which displays information in web dashboards and mobile applications.

The main goal of the system is to support transport authorities in monitoring approved bus routes, detecting route deviations, improving off-peak service availability, and making fair permit fee and incentive-related decisions. The system provides bus owners and drivers with trip visibility through schedule updates and permit fee information and incentive details and breakdown reporting and operational notifications. The system provides indirect benefits to passengers through its creation of better service reliability and improved off-peak bus availability and safer transport operations.

The system performs its main functions through processing IoT-based GPS and sensor events which enable it to detect trip start and end points while identifying route deviations and classifying trips into policy-based time slots and executing automated schedule adjustments and calculating distance-based revenue and off-peak incentives and tracking unsafe driving events and issuing alerts and generating operational and financial reports.

The system serves four main user groups which include transport authority officers system administrators private bus owners and bus drivers. Transport authority officers use the web dashboard to monitor bus operations which includes tracking route compliance and schedule adherence and reporting violations and revenue-related information. The mobile application allows bus owners and drivers to access their assigned schedules and trip summaries and earnings and permit fee details and incentives and alerts. System administrators handle user management and route administration and bus management and the establishment of policy rules and system configuration and the definition of access rights.

2. Overall Description

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System is an integrated software platform connected with IoT-based on-board bus monitoring devices. The system enables Sri Lankan public bus services to carry out digital monitoring processes together with scheduling tasks, compliance checking operations, revenue calculation activities, and operational reporting functions. The system gathers operational data from IoT devices, which includes bus identity information, GPS location data, timestamp records, route event details, speed measurements, distance travelled, and sensor-based movement data, and then it processes this information through its centralized backend system.

The system provides a common platform for transport authorities, bus owners, drivers, and system administrators. Transport authorities use the system to monitor approved bus routes, identify route deviations, review bus availability, check service coverage, and support permit-related decisions. Bus owners and drivers use the mobile application to view schedules, trip details, earnings-related information, permit fee estimates, incentives, alerts, and breakdown reporting options. The system enables system administrators to control all aspects of user management and vehicle handling, route creation, policy enforcement, and system setup processes.

The proposed system is different from a basic bus tracking system because it does not only display bus locations. The system uses event-driven bus monitoring together with policy-based time-slot classification, automated schedule adjustment support, distance-based revenue calculation, off-peak incentive handling, driver behavior monitoring, and dashboard-based reporting. The system provides support for operational management together with regulatory decision-making functions.

The system depends on several major software and hardware elements which include IoT devices installed in buses, a backend server, a database, web dashboards, mobile applications, and communication services, and map-based route data. The on-board device captures bus movement and operational events while the backend processes this data and stores it for monitoring analysis reporting, and calculation purposes. The dashboard and mobile application provide user-friendly access to the processed information.

The software is expected to improve transport operation visibility, reduce dependency on manual schedules and phone-based reporting, support fair permit fee and incentive calculations, and help transport authorities make evidence-based decisions. The system enables

transport authorities to communicate effectively with bus operators through digital interfaces which deliver verified operational data.

The system functions as a smart transport management platform which unites IoT-based data collection with software systems that handle monitoring, scheduling, revenue management, and reporting. The system exists to enhance public bus service reliability by improving route compliance, off-peak service availability, and operational transparency, and data-driven transport governance. The project proposal also identifies the need for GPS-based monitoring, route deviation detection, automated scheduling, distance-based revenue calculation, and mobile updates for bus owners and drivers.

2.1 Product Perspective

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System functions as a complete transport management system which enables public bus operations through its route tracking capabilities, schedule management features, revenue assessment tools and operational documentation functions. The system functions as a dual-purpose solution which provides both regulatory solutions and operational assistance for transport authorities, private bus owners and drivers.

Current public transportation systems together with commercial fleet tracking systems emphasize vehicle monitoring in real time while providing arrival time estimates, passenger details, route mapping and overall fleet supervision. The systems provide advantages to passengers through their convenience features, which enable basic operational monitoring, but they fail to meet all requirements for policy enforcement and distance-based revenue assessment of permits and off-peak service rewards and automated schedule modifications and communication between owners and regulators. The project proposal identifies that many existing intelligent transportation systems mainly support passenger experience, journey planning, and logistics, while fixed permit revenue models, off-peak service gaps, and operator-side visibility remain weak areas.

The new system offers broader software capabilities compared to existing GPS tracking systems. The system displays the present location of a bus. The system receives operational events as GPS and sensor data from IoT devices which it processes through a central backend system to verify route adherence while categorizing trips into specific time periods and enabling schedule modifications and calculating permit costs and driver bonuses and tracking

dangerous driving patterns and showing results through web-based dashboards and mobile applications.

The system also improves upon manual transport authority processes. The current system requires route monitoring and bus availability checks and permit decisions and schedule management to rely on paper schedules and phone calls and inspections and reports that arrive late. The system creates transportation authorities with restricted information which makes it hard to determine whether buses follow selected routes and maintain correct schedules and operate during non-peak times. The proposed system provides verified digital data that can support evidence-based decisions.

The system includes four software components that work together as a complete product. The Event-Driven Bus Monitoring Engine collects data from bus movement events and route compliance tracking. The Policy-Driven Bus Scheduling and Time-Slot Management Engine uses transport rules to create service period classifications which enable automatic schedule changes. The Distance-Based Revenue Management Component calculates permit fees and off-peak incentives using actual operational distance and time-slot data. The IoT-Integrated Driver Monitoring and Operational Reporting Mobile Application provides trip updates, driver behavior alerts, earnings details, and operational reports for bus owners and drivers.

The system depends on IoT devices installed in buses which connect to a backend processing layer that stores data in a PostgreSQL database while using map and route data to operate a web dashboard for transport authorities and a mobile application for bus owners and drivers. The IoT device captures GPS, speed, timestamp, acceleration, and gyroscope-related data, while the backend processes this data and shares relevant outputs with the dashboard and mobile application. The proposal includes a centralized server design that enables system functions to share data for monitoring purposes and revenue calculation and mobile app updates.

The system functions as a complete system to monitor buses instead of being just a bus tracking application. The system operates as a smart transport governance platform which integrates eight different components: IoT monitoring, route compliance checking, policy-based scheduling, revenue management, driver monitoring, and stakeholder reporting. The product meets the needs of transport authorities who require three specific operational capabilities: they need to achieve transparency, make equitable decisions about permit revenue, and improve their service planning during off-peak hours their public bus service operations.

2.1.1 System Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System functions across multiple operating environments, which include backend server environment and web browser environment, mobile operating systems, and embedded IoT device runtime environment. The system operates through standard operating system services, which provide networking and storage, time synchronization, security certificates, and application runtime support without requiring users to obtain special low-level operating system control.

The backend system needs to operate in a server environment that includes Linux and Windows Server. The backend uses operating system network services to receive IoT event data, serve REST API requests, and communicate with the database and support dashboard and mobile application communication. The server operating system must enable application logging, environment configuration, and scheduled services, secure communication, and database connectivity.

The web dashboard interface shall be accessed through a modern web browser running on common desktop operating systems such as Windows, Linux, or macOS. Transport authority officers and system administrators need only a supported browser and internet access to use the dashboard, without any requirement for desktop software installation.

The mobile application shall run on Android and/or iOS mobile operating systems. The mobile operating system shall provide standard services such as internet connectivity, user authentication support, push notifications, local temporary storage, and secure communication with the backend server. The mobile application serves as a primary tool for bus owners and drivers to access their schedules, trip information, alerts, earnings, permit fee details, incentives, and operational notifications.

The IoT device firmware shall run on the ESP32 microcontroller environment using the Arduino framework or an equivalent embedded development environment. The embedded device connects with GPS and motion sensors to handle specific operational events, which it sends to the cloud backend system. The device depends on embedded runtime services to handle Wi-Fi communication and to read serial data from the GPS module, to manage sensor communication and timestamp processing, and to store temporary data when necessary.

The system requires precise system time information to function properly between its backend components and IoT event sources. Correct timestamps are important for trip monitoring, route event ordering, time-slot classification, schedule adjustment, revenue calculation, and off-peak incentive processing.

The system connects to operating environments through its standard network functions, storage functions, security functions, timing functions, browser functions, mobile functions, and embedded runtime functions. Normal users need no special operating system customization, but backend server and IoT device systems require a correct setup for dependable data transmission , processing, and monitoring functions.

2.1.2 User Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System provides role-based user interfaces for transport authority officers, system administrators, bus owners, and bus drivers. The system includes a web-based dashboard for transport authorities and administrators, and a mobile application for bus owners and drivers.

The system's main user interface principles include its design for simplicity and clarity together with its role-based access system and real-time visibility features, which support simple decision-making processes. The system will be used by both technical and non-technical users because its interfaces use clear labels together with simple navigation, dashboard summaries, and map-based views, alert indicators, and report-based outputs. The interfaces shall avoid unnecessary complexity and display only the information relevant to each user role.

The transportation authority web dashboard will deliver an overview of public bus operations through its main operational display. The system will display bus monitoring for each route together with real-time and scheduled bus location tracking, route deviation notifications, bus operational status, off-peak service area coverage, compliance assessment with regulations, timetable details, financial summaries, and incident violation documentation. The system will present bus routes through map-based visualization, which shows current bus locations together with route deviations and tracked operational activities.

System administrators will gain system access through the system administrator interface, which enables them to manage system configuration and data management tasks. The system allows for user management, together with bus and route handling, driver assignment, and transport authority officer management and policy rule creation, time-slot definition, system

parameter adjustment, and access permission setup. The administrator interfaces will enable users to perform create, read update, and delete functions whenever necessary.

The mobile application for bus owners will show all operational details together with financial information. Bus owners will access complete information about their registered buses, assigned schedules, and trip details which include distance records, permit fee calculations, and off-peak bonuses and earnings information, system notifications, and failure analysis reports. Bus owners who require operational information during their daily bus operations will benefit from the mobile interface, which provides them with immediate access to system data.

The driver mobile application shall provide schedule details, trip status, route-related alerts, unsafe driving alerts, and breakdown reporting options. Drivers will access their scheduled trips through the system, which allows them to change their trip status, receive notifications, and report any operational problems, including bus breakdowns. The mobile interface shall use simple buttons which display readable text together with alert messages to decrease confusion during active operations.

Interface	**Main Users**	**Purpose**
Login Interface	All users	Secure access to the system based on user role
Authority Dashboard	Transport authority officers	Monitor routes, buses, alerts, schedules, compliance, and revenue summaries
Route Monitoring Map	Transport authority officers	View bus movement, route compliance, and deviation events
Policy and Time-Slot Management Interface	Transport authority officers/administrators	Define time slots, policy rules, and schedule-related configurations
Schedule Management Interface	Transport authority officers / bus owners	View and manage bus schedules and operational assignments
Revenue and Incentive Report Interface	Transport authority officers / bus owners	View permit fee, distance-based revenue, and off-peak incentive details

Alert and Violation Interface	Transport authority officers / bus owners / drivers	View route deviation, speeding, unsafe driving, breakdown, and compliance alerts
Owner Mobile Dashboard	Bus owners	View bus operations, trip summaries, earnings, fees, and incentives
Driver Mobile Dashboard	Drivers	View assigned trips, alerts, trip status, and breakdown reporting options

Table 1 User Interface

2.1.3 Hardware Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System needs multiple hardware interfaces to gather, send, process, and present public bus operational information. The software system connects with on-board IoT hardware, which bus systems use, and with backend/cloud infrastructure and network devices and user access devices that include computers and mobile phones.

The main hardware interface of the system is the on-board IoT device installed inside each registered bus. The device gathers movement and operational information about the bus, which it transmits to the backend system for processing. The IoT device shall be uniquely associated with a registered bus so that all received data can be mapped to the correct bus, route, trip, driver, and owner.

The following special hardware components are required for software operation:

Hardware Component	Purpose / Interface with Software
ESP32 Microcontroller	Acts as the main on-board processing unit. It reads GPS and sensor data, performs basic event processing, and transmits data to the backend system.
NEO-6M GPS Module	Provides GPS coordinates, speed-related data, and timestamp information used for bus location tracking, trip monitoring, route compliance checking, and distance calculation.
MPU6050 IMU Sensor	Provides acceleration and gyroscope data used for driver behavior monitoring, such as harsh braking, rapid acceleration, sharp turning, and abnormal movement detection.
Vehicle Power Supply / Voltage Regulator	Supplies stable power to the IoT device from the bus electrical system.
GPS Antenna / Sensor Connections	Supports accurate GPS signal reception and sensor communication between the IoT device and connected modules.
Android / iOS Mobile Device	Used by bus owners and drivers to access the mobile application for schedules, trip details, alerts, reports, permit fee details, and incentives.
Desktop or Laptop Computer	Used by transport authority officers and system administrators to access the web dashboard through a browser.
Wi-Fi / 4G Communication Module or Network Interface	Allows the IoT device to transmit operational events and sensor data to the cloud/backend system.

Table 2 Hardware Interfaces

2.1.4 Software Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System interfaces with multiple software products and platforms and frameworks, services, and APIs to enable the ingestion of IoT data and backend processing and database storage, dashboard visualization, mobile access, and map-based monitoring and reporting functions.

The backend system will function as the primary software connection point for all systems. The system will process IoT event data while executing route monitoring procedures, determining revenue and incentive calculations, and user access control, and delivering data through REST APIs to both the web dashboard and mobile application.

The following software interfaces are required for the system:

Software / Service	**Purpose / Interface with the System**
React JS	Used to develop the transport authority and administrator web dashboard. It interfaces with the backend through REST APIs to display bus locations, route compliance, alerts, schedules, policies, revenue summaries, and reports.
React Native	Used to develop the mobile application for bus owners and drivers. It interfaces with backend services to display schedules, trip summaries, earnings, permit fee details, incentives, alerts, and breakdown reporting options.
Spring Boot	Used as the backend REST API and business logic layer. It processes IoT events, handles authentication, manages system data, applies policy rules, performs revenue calculations, and connects frontend, mobile, IoT, and database services.
PostgreSQL	Used as the primary relational database. It stores structured data such as users, buses, routes, drivers, owners, schedules, policy rules, IoT events, trip records, revenue records, alerts, and audit logs.
Azure IoT Hub or Equivalent IoT Ingestion Service	Used to receive data packets from IoT devices installed in buses. It supports secure communication between IoT devices and the backend system.

Leaflet / Mapbox GL JS or Equivalent Map Library	Used to display digital maps, approved route polylines, bus locations, halts, junctions, geofences, route deviations, and alerts in the dashboard and mobile application.
External Map / Road API	Optionally used for road-snapping, route polyline verification, distance support, and map-based validation where required.
Authentication/ Authorization Service	Used to manage secure login, role-based access, user sessions, and protected access to dashboard and mobile functions.
Reporting / Export Service	Used to generate operational, compliance, schedule, revenue, incentive, and driver behavior reports in formats such as PDF, Excel, or CSV, where feasible.

Table 3 Software Interfaces and purpose

2.1.5 Communication Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System uses multiple communication interfaces to transmit information between the on-board IoT device and cloud/backend services and database, web dashboard, mobile application, map services, and notification services. The system requires these communication interfaces to enable real-time bus monitoring, bus status checks, bus schedule tracking, revenue calculation, alert generation, and report generation.

The main communication flow starts from the IoT device installed in the bus. The IoT device collects GPS, timestamp, speed, acceleration, gyroscope, and operational event data and transmits selected data packets to the cloud/backend system. The system shall support event-based communication instead of unnecessary continuous data streaming, so that data transfer cost and bandwidth usage can be reduced.

The following communication interfaces are required for the system:

Communication Interface	Description
IoT Device to Cloud/Backend Communication	The on-board IoT device shall transmit GPS and sensor-based event data to the cloud/backend system using Wi-Fi or 4G connectivity. MQTT or HTTPS may be used for secure IoT data transmission.
IoT Ingestion Service Communication	Azure IoT Hub or an equivalent IoT ingestion service shall receive device data and forward it to the backend processing layer.
Web Dashboard to Backend Communication	The transport authority web dashboard shall communicate with the backend system through secure REST APIs over HTTPS.
Mobile App to Backend Communication	The bus owner/driver mobile application shall communicate with the backend through secure REST APIs over HTTPS to retrieve schedules, trip details, alerts, earnings, permit fee details, incentives, and reports.
Backend to Database Communication	The backend system shall communicate with the PostgreSQL database using database connectivity such as JDBC to store and retrieve users, buses, routes, trips, policies, IoT events, schedules, alerts, and revenue records.
Backend to Map Service Communication	The backend or frontend shall communicate with map services such as Leaflet, Map box, or external road/map APIs to display route polylines, bus locations, halts, junctions, and route deviation information.
Notification Communication	The system shall send alerts and notifications to relevant users through the web dashboard and mobile application. These alerts may include route deviations, speeding, unsafe driving behavior, schedule changes, breakdowns, and policy violations.
Authentication Communication	User login and session handling shall be performed through secure backend authentication APIs. Tokens such as JWT may be used to protect dashboard and mobile app requests.
Offline / Delayed Data Synchronization	If network connectivity is temporarily unavailable, the IoT device or mobile application may temporarily store operational data and synchronize it with the backend once connectivity is restored.

Table 4 Communication Interfaces

2.1.6 Memory Constraints

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System uses memory and storage across the on-board IoT device, backend server, database, web dashboard, and mobile application. The on-board IoT device has limited internal memory, which enables it to store only temporary data that includes recent GPS readings and timestamps, sensor values, event flags, and unsent event packets. The system prohibits the storage of extensive past data records. The device will store particular event information when there is no network connection, but it will send these details to the backend after the connection is available again.

The backend server needs enough memory capacity to handle IoT events, route compliance checks, time-slot classification and schedule adjustment, revenue calculation, alerts, API requests, and report generation. The database shall provide external storage for long-term data such as users, buses, routes, schedules, policies, trip logs, IoT events, revenue records, alerts, violations, and audit logs. The web dashboard and mobile application shall use memory efficiently by loading only required data, using filtering, pagination, and summaries, and temporary local storage. Sensitive operational or financial data shall not be stored locally unless it is required and securely protected.

The system shall follow the following memory-related constraints:

System Area	Memory / Storage Constraint
IoT Device	Shall use limited internal memory for temporary GPS, sensor, and event data only.
IoT Data Buffer	Shall temporarily store unsent event packets when connectivity is unavailable.
Backend Server	Shall have enough runtime memory to process concurrent IoT events, API requests, calculations, and reports.
Database Storage	Shall store long-term operational, route, policy, trip, revenue, alert, and user data.
Web Dashboard	Shall use optimized data loading, filtering, and pagination to avoid high browser memory usage.
Mobile Application	Shall store only necessary temporary data and avoid unnecessary local storage of sensitive information.
Reports and Logs	Shall be archived or summarized periodically to control database growth.

Table 5 Memory Constraints

2.1.7 Operations

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System requires both normal operations and special operations from different user groups, such as transport authority officers, system administrators, bus owners, and drivers.

Normal Operations

Normal operations are the regular activities performed by users during daily system usage.

User Role	Normal Operations
System Administrator	Login to the system, manage user accounts, assign user roles, manage buses, manage routes, configure system settings, and monitor system status.
Transport Authority Officer	View authority dashboard, monitor route-wise bus operations, check bus locations, review trip records, view route compliance status, manage policy rules, view schedules, and generate operational reports.
Bus Owner	Login to the mobile application, view registered buses, check assigned schedules, view trip summaries, view distance records, check permit fee details, view incentive details, and receive operational alerts.
Driver	Login to the mobile application, view assigned trips, update trip status where required, receive route and schedule alerts, report breakdowns, and view driver-related notifications.
System	Receive IoT event data, process GPS and sensor data, detect trip events, classify time slots, update dashboards, calculate revenue/incentives, generate alerts, and store data in the database.

Table 6 Normal Operations

Special Operations

Special operations are performed during exceptional situations, system issues, operational disruptions, or administrative needs.

Situation	Special Operation
Route Deviation	The system shall detect route deviation events and notify relevant authority users and bus owners.
Bus Breakdown	Drivers or bus owners shall be able to report breakdowns, and the system shall support schedule adjustment or reassignment where possible.
Off-Peak Service Monitoring	The system shall identify whether buses are operating during off-peak or nighttime slots and support incentive-related calculations.
Unsafe Driving Event	The system shall generate alerts for events such as over speeding, harsh braking, sharp turning, or abnormal movement.
Network Failure	The IoT device or mobile application may temporarily store selected event data and synchronize it with the backend when connectivity is restored.
Policy Change	Authorized officers shall be able to update time-slot definitions, rate rules, incentive rules, and scheduling policies.
Manual Review	Authority officers shall review flagged route deviations, missing trips, abnormal operations, revenue records, or disputed events.
Report Generation	Authorized users shall generate route-wise, bus-wise, schedule-wise, compliance-wise, revenue-wise, and incentive-related reports.
Security Operation	The system administrator shall manage access permissions, reset user accounts, and monitor unauthorized access attempts.

Table 7 Special Operations

2.1.8 Site Adaptation Requirement

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System will operate according to the transport authority requirements, the chosen bus routes and bus fleet, and the specific operational settings of the system deployment. The system must support site-specific configurations because public bus operations differ according to provincial, routing, timing, and authority operational procedures.

The system requires installation site-specific adaptations, which include route information, approved route paths, and bus stop locations, registered bus details and owner and driver information, operating schedules, time-slot specifications, permit fee structures, incentive rates, and policy guidelines. The system will establish user access rights together with operational roles for transport authority personnel and system managers, bus operators, and drivers.

The IoT devices shall be installed and mapped to the correct bus registration number or unique bus ID. The system testing requires each device to undergo assessment for GPS signal reception capabilities, sensor accuracy, and power supply stability, and network connectivity before the system starts its operational phase. The system will be implemented on selected pilot routes as the first step, which will result in the creation of Route maps and bus details, and user information and policy settings to be established during the first system setup.

The backend server, database, web dashboard, mobile application, IoT data ingestion service, and map services require deployment environment specific configuration. The system needs to execute server URL configuration while executing database connection settings and API endpoints, and map keys and notification settings, backup settings, and security settings. The system shall support future expansion by allowing additional routes, buses, users, policies, and time slots to be added without redesigning the full system.

The installation site needs to provide continuous internet service that supports both the authority dashboard and backend services, and mobile application use. Buses need to establish appropriate communication links through Wi-Fi and 4G systems to enable IoT event data transmission. The system will handle temporary network interruptions through its ability to perform delayed synchronization when this function is available.

The site-specific testing process needs to be conducted before final deployment to test all system functions, which include route monitoring, time-slot classification, route deviation

detection, schedule adjustment, revenue calculation alerts, dashboard reports, and mobile app functions. The proposal requires GPS-based monitoring together with automated scheduling, distance-based revenue calculation, and mobile updates and authority dashboards to function as essential system components.

2.2 Product Functions

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System functions as an integrated system which enables public bus tracking and route monitoring, automatic schedule creation, revenue assessment and driver tracking and operational reporting. The system itself provides major operational capabilities which are listed in the following summary.

The system requires transport authority officers and system administrators and bus owners and drivers to authenticate their identity before accessing system functions according to their designated access rights. Each user shall access only the functions relevant to their role. System administrators are responsible for controlling user access to system resources which include user accounts and buses and routes and drivers and owners and policy settings and system access rights.

The system enables management functions for buses and routes and bus owners and drivers. Registered buses will establish connections with bus owners and drivers as well as approved routes and IoT devices. Route information will contain route numbers and route names and approved route paths and bus halts and junctions and schedules and operating policies that are associated with the route.

The system will use Internet of Things technology to collect bus events through GPS and sensor data from on-bus Internet of Things devices. The system will obtain essential operational information which includes bus identification numbers GPS location data time stamps vehicle speed and distance measurements and acceleration and gyroscope sensor data and trip and stop events and selected driver performance indicators. The proposal identifies GPS devices and sensors as the main method for collecting bus location, speed, distance, and driver behavior data.

The system shall provide event-driven bus monitoring. The system will handle essential events through its processing system which includes trip start, trip end, halt detection, junction entry/exit, route deviation, offline status, and abnormal movement together with continuous

GPS streaming. The system helps to decrease unnecessary data transmission while still enabling transport monitoring and decision-making capabilities.

The system shall support route compliance monitoring and deviation detection. The system will assess bus movement data against authorized route details to determine whether a bus operates within its designated route. The system will create alerts for designated users when it detects either route deviation or unauthorized route movement.

The system requires policy-based control to manage both scheduling and time-slot allocation. The system will determine bus trip classification according to established operational time frames, which include peak time, normal time, off-peak time and night time periods based on policy rules and trip timestamps. The system enables schedule modifications during operational interruptions, which include delays and missing trips and equipment failures.

The system will enable automatic monitoring of schedule changes and service availability. Transport authority users will access route-specific bus availability information while they track service disruptions and verify whether essential services operate during off-peak and night time hours. This project aims to enhance public bus service availability which operates during periods of reduced service.

The system enables calculation of revenue and permit fees according to distance traveled. The system employs approved trip distance and route information together with time-slot regulations to determine permit-related revenue. The system enables bus operators to receive off-peak and night-time bonuses when they operate services during times of low passenger demand. The proposal explains that permit revenue is calculated using GPS-based distance data and that commissions are used to encourage off-peak bus operations.

The system requires driver behavior monitoring system which uses speed data and acceleration data and gyroscope data and timestamp data for its operations. The system will detect specific hazardous and unusual driving patterns which include overspeeding and harsh braking and sharp turning and rapid acceleration and unusual slow movement. The system will deliver relevant alerts and reports through the dashboard and mobile application.

The system enables transport authorities and administrators to access web dashboard functions. The dashboard displays monitoring data for each route together with real-time bus locations and route compliance details and alert notifications and schedule information and time-slot

coverage and revenue summaries and incentive records and violation records and operational reports.

The system enables bus owners and drivers to access mobile application functions. Bus owners can monitor their bus operations through viewing trip summaries and schedules and distance logs and permit fee estimates and incentive details and earnings-related information and alerts and breakdown reports. Drivers can access their assigned trips which include current trip status and route alerts and schedule updates and unsafe driving alerts and breakdown reporting options.

The system must deliver alerting and notification capabilities to track vital operational activities. The alerts system will report on route deviations and speeding violations and unsafe driving instances and missed trips and vehicle breakdowns and breaches of policy and alterations in the schedule and updates about system performance. Transport authorities and owners and drivers can make quicker decisions because these alerts provide them with vital information.

The system will offer users functions to create reports and perform data analysis. Authorized users shall generate route-wise, bus-wise, schedule-wise, compliance-wise, revenue-wise, incentive-wise, and driver behavior reports. The reports will enable organizations to monitor their operations and enforce policies and review their financial performance and develop public transport plans for future use.

The product enables IoT-based bus data collection to connect with software programs that handle monitoring and scheduling, revenue management, and driver oversight, alert systems, and display dashboards and mobile reporting. The system provides a comprehensive solution for smart public transport management, which goes beyond basic bus tracking functions.

2.3 User Characteristics

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System will be used by different user groups with different responsibilities, technical knowledge, and operational needs. The primary system users include transport authority officers system administrators bus owners and bus drivers. Passengers are indirect beneficiaries of the system because they benefit from improved service reliability, route compliance, off-peak availability, and safer bus operations. The project proposal identifies government authorities, bus owners, bus drivers, and passengers as the main stakeholders of the proposed system.

User Type	User Characteristics
System Administrator	A technical or semi-technical user responsible for managing system configurations, users, buses, routes, access permissions, and system data. This user is expected to have good computer literacy and basic knowledge of web-based administration systems.
Transport Authority Officer	A non-technical or moderately technical user who uses the web dashboard to monitor bus operations, route compliance, schedules, alerts, revenue reports, and service availability. This user may be familiar with office software, reports, spreadsheets, and existing transport authority procedures.
Bus Owner / Fleet Operator	A business-oriented user who uses the mobile application to view registered buses, schedules, trip summaries, distance logs, permit fee estimates, incentive details, alerts, and operational reports. This user may have basic to moderate mobile application usage experience.
Bus Driver	A field-level user who uses the mobile application to view assigned trips, receive route or schedule alerts, update trip status where required, and report breakdowns. This user may have basic mobile phone knowledge, so the interface should be simple, clear, and easy to use
Passenger	An indirect user of the system. Passengers do not directly manage the system, but they benefit from improved off-peak bus availability, better route reliability, and safer transport services.

Table 8 User Characteristics

2.4 Constraints

The system development process requires multiple technical and operational constraints as its basic foundation. The GPS system experiences accuracy changes which depend on various factors including signal strength, weather conditions, road environment, and bus movement. Real-time data transmission becomes impossible because some areas experience unstable Internet connectivity through Wi-Fi or 4G networks. The IoT device must work with limited processing power, memory, and vehicle power supply, therefore the system should use event-based data transmission and avoid unnecessary continuous data streaming.

The system requires bus schedules and official route data, halt/junction locations, transport policies, and permit fee rates and incentive rules to function correctly. Incomplete or outdated information will hinder route compliance monitoring and time-slot classification, automated scheduling, and revenue calculation processes. The system requires maintenance of security and privacy, role-based access control, and encrypted communication because it manages location data and driver behavior data, user accounts, and financial records. Development choices are also constrained by the selected technologies, such as ESP32, GPS module, sensors, Spring Boot, PostgreSQL, React, React Native, and cloud/IoT services identified for the project.

2.5 Assumptions and Dependencies

All registered buses in the system will operate with functioning IoT devices that have GPS capabilities and necessary sensors, and each device will match a specific bus ID. The system needs buses to maintain continuous power and establish network connections through Wi-Fi or 4G for event data transmission to the backend system. The system stores selected event data during periods of lost connectivity, which will later synchronize with the main system.

The system requires precise current information from transport sources, which include authorized bus routes and route polylines and halt or junction locations, bus schedules and time-slot definitions, permit fee rates, incentive rules, and transport authority policies. The reference data accuracy determines the success of route compliance monitoring and schedule adjustment, time-slot classification, and revenue calculation processes.

The selected software platform environment which includes backend server, PostgreSQL database, and IoT data ingestion service, web dashboard, mobile application, and map services, determines system functionality. The authorized transport authority officers system

administrators, bus owners, and drivers will use the dashboard and mobile application because they will have access to appropriate devices and internet connectivity. The proposal identifies ESP32, GPS module, sensors, Azure IoT Hub, Spring Boot, PostgreSQL, React, React Native, and map services as the main technologies for the system. The system future versions will require external government transport databases, SMS and push notification services, advanced analytics modules, and larger fleet deployment environments for operation. The system requires a modular design which enables the addition of routes, bus policies, reports, communication methods, and external integrations without complete redesign.

2.6 Apportioning of Requirements

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System will execute its requirements through multiple stages which will follow the established order of priorities, system dependencies, and system integration requirements. The first implementation will focus on core data management together with backend services because all other modules need access to user and bus, route, schedule, and policy, and database data. The system will begin to implement IoT event receiving and route monitoring functions when the development of both backend system and database system reaches completion. The team will create the policy/time-slot engine and revenue calculation module, a dashboard, and mobile application for subsequent integration with the project.

Phase	Requirements to be Implemented	Priority
Phase 1: Core System Setup	User authentication, role-based access, bus management, route management, driver/owner management, database setup, and backend API structure.	Essential
Phase 2: IoT Event Monitoring	GPS event receiving, trip start/end detection, halt/junction event processing, route compliance checking, and route deviation alerts.	Essential
Phase 3: Policy and Scheduling Engine	Time-slot classification, policy rule management, schedule generation, service availability checking, and schedule adjustment during delays or breakdowns.	Essential
Phase 4: Revenue and Incentive Module	Distance-based permit fee calculation, off-peak/night incentive calculation, revenue summaries, and financial reports.	Essential
Phase 5: Dashboard and Mobile Application	Transport authority dashboard, owner mobile app, driver mobile app, alerts, reports, trip summaries, earnings details, and breakdown reporting.	Essential
Phase 6: Integration and Testing	Integration of IoT monitoring, policy engine, revenue module, dashboard, and mobile application; functional testing, performance testing, validation, and final improvements.	Essential

Phase 7: Future Enhancements	Advanced analytics, SMS alerts, external government system integration, forecasting, and large-scale province-wide deployment support.	Desirable / Optional

Table 9 Apportioning of Requirements

3. Specific Requirement (DD)

3.1 External Interface Requirements

3.1.1 User Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System will deliver different user interfaces that match the specific needs of system administrators, transport authority officers, bus owners, and bus drivers. The system will provide a web-based dashboard which transport authority and administrator users can access, and a mobile application that bus owners and drivers can use. The interfaces will enable users to monitor routes and schedule buses, manage policies, calculate revenue, monitor drivers, receive alerts, and create operational reports.

The user interfaces shall follow simple and clear design principles because the system will be used by both technical and non-technical users. The interfaces will display readable text which users can navigate through while viewing dashboard summaries, map-based visualizations and alert indicators, tables, filters, and action buttons. The system will show users only the functions that are permitted for their specific user roles.

Interface ID	User Interface	Main Users	Description
UI-01	Login Interface	All users	Allows users to securely log in to the system using username/password or assigned credentials. The system shall validate login details and redirect users based on their role.
UI-02	Admin Dashboard	System Administrator	Allows administrators to manage users, roles, buses, routes, drivers, bus owners, IoT device records, and system configurations.
UI-03	Transport Authority Dashboard	Transport Authority Officers	Displays route-wise bus operations, bus availability, route compliance status, alerts, schedule summaries, revenue summaries, and system performance indicators.

UI-04	Route Monitoring Map	Transport Authority Officers	Shows approved route paths, bus locations, halt/junction points, route deviation events, and trip progress using a map-based interface.
UI-05	Policy and Time-Slot Management Interface	Transport Authority Officers / Admin	Allows authorized users to define and update time slots such as Morning Peak, Off Peak, Office Peak, and Night Time. It shall also allow policy rule configuration for scheduling, grace periods, penalties, and incentive periods.
UI-06	Schedule Management Interface	Transport Authority Officers	Shows route-wise bus assignments, confirmed buses, unavailable buses, time-slot distribution, schedule gaps, and automatic schedule adjustment suggestions.
UI-07	Revenue and Incentive Interface	Transport Authority Officers / Bus Owners	Displays distance-based permit fee calculations, off-peak incentive details, daily/monthly revenue summaries, and financial reports.
UI-08	Bus Owner Mobile Dashboard	Bus Owners	Allows bus owners to view registered buses, trip summaries, assigned schedules, earnings-related information, permit fee estimates, incentives, and driver-related alerts.
UI-09	Report on the Generation Interface	Transport Authority Officers / Admin	Allow authorized users to generate route-wise, bus-wise, schedule-wise, revenue-wise, incentive-wise, and violation reports

Table 10 User Interfaces Requirements

3.1.2 Hardware Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System interacts with several hardware components to collect, process, and transmit bus operation data. The primary hardware interface for the system operates through an onboard IoT device which exists inside every registered bus. The device gathers GPS location data together with speed timestamps and acceleration data and gyroscope data and movement-related information. It transmits selected event data to the backend system which handles route monitoring and scheduling and revenue calculation and driver monitoring and reporting functions.

The hardware interfaces of the system are described below:

Hardware Component	Interface with Software Product
ESP32 Microcontroller	Acts as the main on-board processing unit. The software/firmware running on the ESP32 reads GPS and sensor data, identifies important events, and transmits data to the backend system.
NEO-6M GPS Module	Provides GPS coordinates, speed, and timestamp data. This data is used by the software for real-time bus tracking, trip monitoring, route compliance checking, distance calculation, and time-slot classification.
MPU6050 IMU Sensor	Provides acceleration and gyroscope data. The software uses this data to detect unsafe or abnormal driving behavior such as harsh braking, rapid acceleration, sharp turning, and abnormal movement.
Vehicle Power Supply / Voltage Regulator	Supplies stable power to the IoT device from the bus electrical system. The software depends on continuous device operation to receive reliable event data.
Wi-Fi / 4G Communication Interface	Allows the IoT device to send event data to the cloud/backend system. This interface supports communication between the moving bus and the server.
GPS Antenna / Sensor Connections	Supports accurate GPS reception and reliable sensor communication. Accurate signal reception is required for route monitoring and distance calculation.

Desktop / Laptop Computer	Used by transport authority officers and system administrators to access the web dashboard through a browser.
Android / iOS Mobile Device	Used by bus owners and drivers to access the mobile application for schedules, alerts, trip summaries, earnings, permit fee details, incentives, and breakdown reporting.

Table 11 Hardware Requirements

The NEO-6M GPS module will establish serial communication with the ESP32 microcontroller. The ESP32 will use I²C to establish communication with the MPU6050 sensor. The onboard device will process the collected data and send it to the backend system through Wi-Fi and 4G connections. The backend system will receive this data which will be stored in the database for further processing and display.

Every IoT device needs a bus ID or bus registration number to establish its unique identity. This requirement allows correct identification of the bus which produced specific GPS events and trip events and route deviations and driver behavior alerts and distance records. The system requires this mapping to establish hardware data connections with bus operations.

The hardware interface must work properly within the active bus operation environment. The IoT device needs to maintain stable power supply and receive GPS signals and read sensors and buffer temporary data and transmit data securely. The device can store specific event packets during network outages which will synchronize with the backend system once connectivity returns.

The software product does not directly control complex vehicle mechanical systems. It only interacts with the installed IoT device and connected sensors to collect operational data. The appendix or hardware design section may contain hardware-level wiring diagrams and circuit designs and casing designs and detailed sensor calibration information if needed.

3.1.3 Software Interfaces

The system will connect with multiple software platforms, frameworks, and services, APIs to enable IoT data collection and backend processing, database management and dashboard visualization, and mobile application access, map-based route monitoring, alerts, and reporting. The software interfaces enable the complete system to function as a unified entity which includes the embedded IoT device, backend system, web dashboard, mobile application, and database.

Software / Platform	Interface with the System
Arduino IDE / Embedded Firmware Environment	Used to develop and upload firmware to the ESP32 microcontroller. The firmware reads GPS and sensor data, identifies events, and sends data to the backend or IoT ingestion service.
Azure IoT Hub or Equivalent IoT Ingestion Service	Receives event data from IoT devices installed in buses. It acts as the communication bridge between the on-board device and backend system.
Spring Boot Backend	Acts as the main backend application. It receives IoT events, processes route compliance, applies policy and time-slot rules, calculates revenue and incentives, manages alerts, handles authentication, and exposes REST APIs.
PostgreSQL Database	Stores structured system data such as users, buses, drivers, owners, routes, schedules, policies, IoT events, trip records, alerts, violations, revenue records, incentive records, and audit logs.
React Web Dashboard	Provides the web interface for transport authority officers and system administrators. It communicates with the Spring Boot backend through REST APIs to display route monitoring, schedules, alerts, revenue summaries, and reports.
React Native Mobile Application	Provides the mobile interface for bus owners and drivers. It communicates with backend APIs to display schedules, trip summaries, alerts, breakdown reports, earnings, permit fee details, and incentive details.

Leaflet / Map box GL JS or Equivalent Map Library	Displays digital maps, approved bus routes, bus locations, halts, junctions, geofences, and route deviation points in the dashboard or mobile application.
Google Maps Roads API or Equivalent Road/Map API	May be used for road-snapping, route polyline verification, and validating bus movement against real road paths.
Authentication and Authorization Service	Provides secure login, session handling, and role-based access control for administrators, transport authority officers, bus owners, and drivers.
Notification / Alert Service	Sends alerts related to route deviations, unsafe driving, breakdowns, schedule gaps, missed trips, and policy violations to dashboards and mobile users.
Reporting / Export Service	Generates operational, route-wise, schedule-wise, revenue-wise, incentive-wise, and violation reports. Reports may be exported in formats such as PDF, Excel, or CSV where required.

Table 12 Software Requirements

The backend system will serve as the primary connection point which links all software interfaces together. The IoT device will transmit event data either to the IoT ingestion layer or to the backend system. The backend system will validate the data which it receives before processing and storing it in PostgreSQL. The React dashboard and React Native mobile application will obtain processed data from the backend through secure REST APIs.

The software interfaces will provide access control, which depends on user roles. Transport authority officers will obtain access to functions which include monitoring and scheduling, as well as policy and revenue, and reporting functionalities. Bus owners will obtain access to their individual bus-related operational and financial data. Drivers will obtain access to their assigned trip details along with alerts and functions which allow them to report breakdowns. System administrators will handle management tasks for users, buses and routes, policies, and system configurations

All software components will use secure, reliable communication methods when those methods are available. Web and mobile applications will connect to the backend through REST APIs, which use HTTPS as their communication protocol. The backend application will control

database access while preventing web and mobile clients from directly accessing it. Secure communication for IoT data transmission will utilize Azure IoT Hub or equivalent services.

3.1.4 Communication Interfaces

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System shall use multiple communication interfaces to exchange data between the on-board IoT device and the cloud/backend system and the database and the web dashboard and the mobile application and the map services and the notification services. The system needs these communication interfaces to enable GPS-based bus monitoring and to check route compliance and to classify time slots and to create automated schedules and to calculate revenue and to monitor drivers and to generate alerts and reporting.

The main communication flow begins from the IoT device installed inside the bus. The device collects GPS coordinates and timestamps and speed and acceleration and gyroscope values and operational events. The device will use Wi-Fi or 4G to send important event data to the cloud/backend system. The project proposal identifies Wi-Fi/4G communication and Azure IoT Hub and Spring Boot backend and PostgreSQL and React dashboard and React Native mobile application as part of the system environment.

Communication Interface	**Description**
IoT Device to Cloud / Backend Communication	The on-board IoT device shall send GPS and sensor-based event data to the backend or IoT ingestion service using Wi-Fi or 4G connectivity.
IoT Device to Azure IoT Hub / IoT Ingestion Layer	The IoT device may communicate with Azure IoT Hub or an equivalent IoT ingestion service to securely send device-generated data packets.
Backend to Database Communication	The Spring Boot backend shall communicate with the PostgreSQL database to store and retrieve users, buses, routes, schedules, policies, IoT events, trips, alerts, revenue records, incentive records, and audit logs.
Web Dashboard to Backend Communication	The React web dashboard shall communicate with the backend through secure REST APIs over HTTPS.
Mobile App to Backend Communication	The React Native mobile application shall communicate with the backend through secure REST APIs over HTTPS to

	retrieve schedules, trip details, alerts, breakdown reports, permit fee details, earnings, and incentives.
Backend / Frontend to Map Service Communication	The system shall communicate with map services such as Leaflet, Map box, Google Maps Roads API, or equivalent services to display route maps, route polylines, halts, junctions, bus locations, and deviation points.
Notification Communication	The system shall send alerts and notifications to relevant users through the web dashboard and mobile application. These alerts may include route deviations, unsafe driving events, breakdowns, missed trips, schedule gaps, and policy violations.
Authentication Communication	User login and role-based access shall be handled through secure backend authentication APIs. Authentication tokens such as JWT may be used to protect user sessions and API requests.

Table 13 Communication Interfaces

3.2 Architectural Design <For Embedded System>

3.2.1 High-Level Architectural Design

The high-level architecture consists of five major layers: IoT device layer, communication layer, backend processing layer, database layer, and user application layer.

The IoT device layer is installed inside each registered bus. It consists of the ESP32 microcontroller and GPS module and IMU sensor and power supply components. The system collects operational data which includes bus identification and GPS coordinates and timestamps and speed and acceleration and gyroscope measurements and operational events which include trip start and trip end and halt detection and route deviation and unsafe driving and breakdown-related events.

The communication layer transfers selected event data from the bus to the cloud/backend system using Wi-Fi or 4G communication. The system uses an event-driven communication method which sends essential events instead of transmitting all GPS data throughout the day. The process decreases bandwidth requirements while decreasing communication expenses and storage demands and power usage.

The backend processing layer receives IoT event data and applies system logic. The system processes incoming data to perform validation checks, route compliance checks, time slot trip classification, schedule adjustments, distance-based permit fee calculations, off-peak incentive calculations, unsafe driving event detection, alert generation, and report preparation. The system uses this layer to make its most important decisions.

The database layer stores structured system data such as users, buses, drivers, bus owners, routes, route polylines, halts, schedules, policy rules, time-slot definitions, IoT events, trip records, revenue records, incentive records, alerts, violations, and audit logs. PostgreSQL serves as the main database system which enables efficient management of structured data.

The user application layer enables users to view system outputs through a web dashboard and mobile application. The web dashboard helps transport authority officers and administrators track information about routes, policies, schedules, revenue, alerts, and reports. Bus owners and drivers use the mobile application to view schedules, trip summaries, earnings-related details, incentives, alerts, and breakdown reporting options.

High-Level Architecture Flow

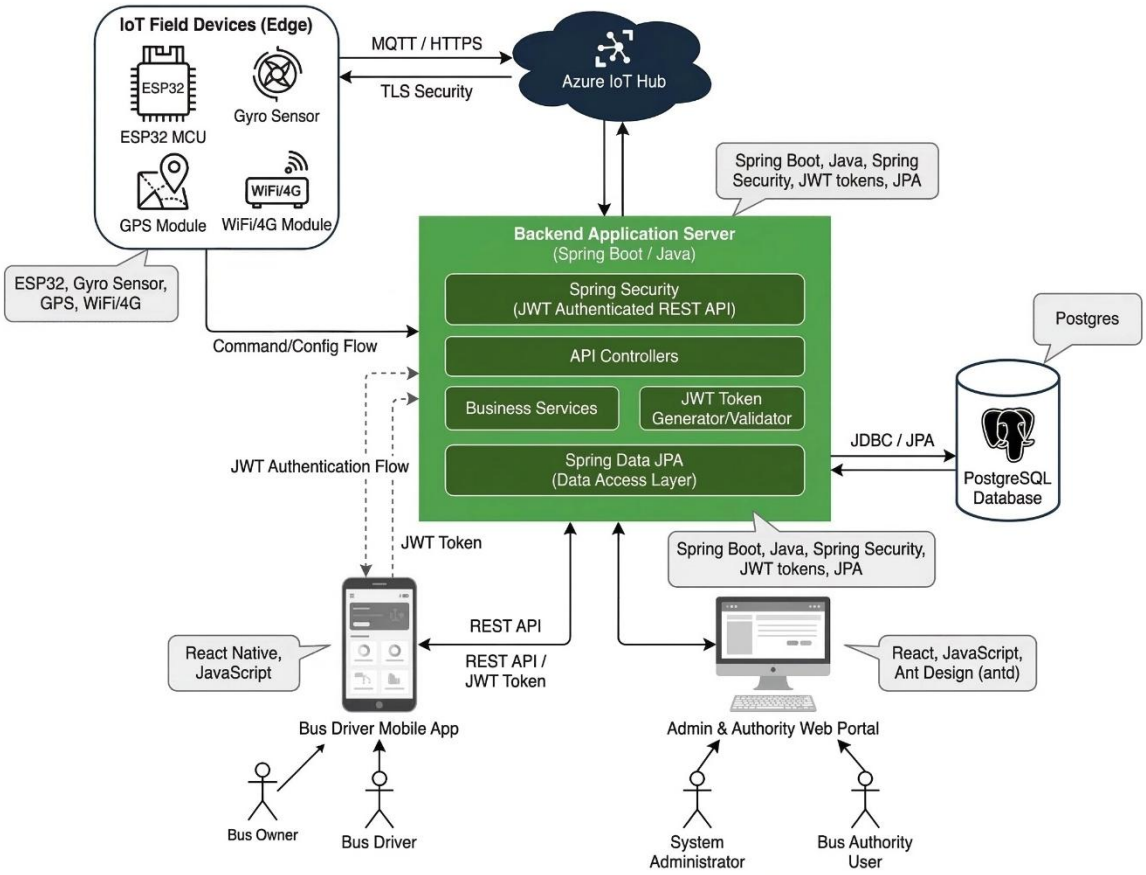


Figure 1 High-Level Architecture Diagram

High-Level Architecture Flow

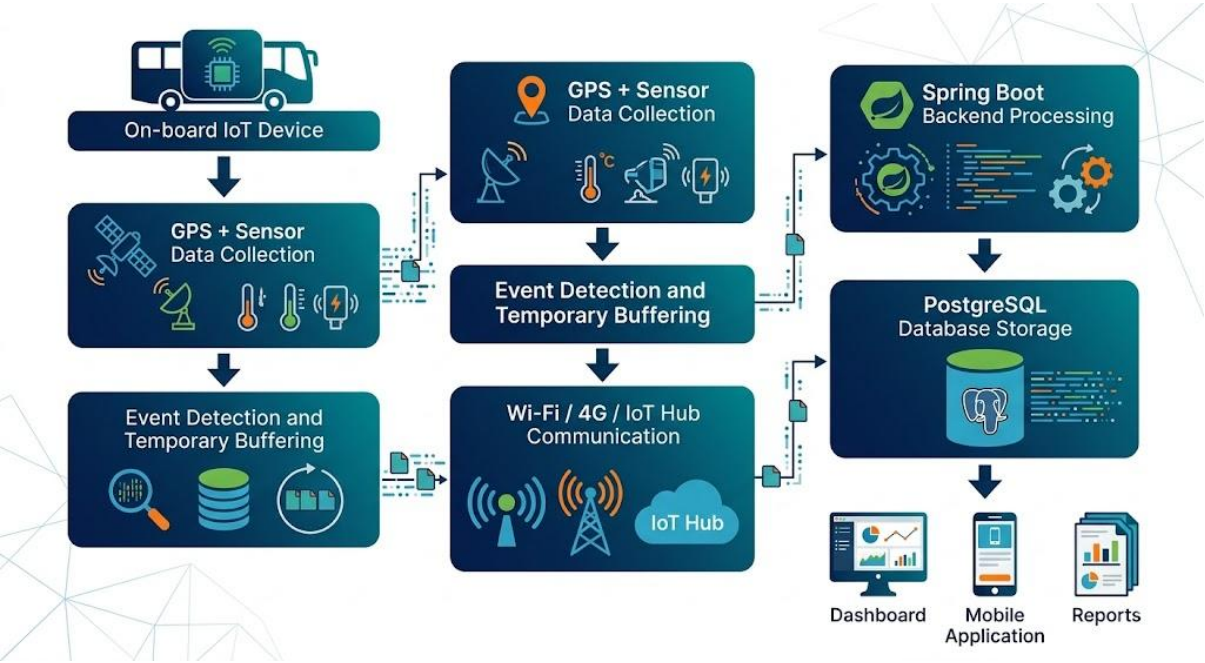


Figure 2 High-Level Architecture Flow

Component Interaction

Layer	Main Function
IoT Device Layer	Collect GPS, speed, timestamp, acceleration, gyroscope, and event data from buses
Communication Layer	Send selected event data from the bus to the backend/cloud system
Backend Processing Layer	Process route compliance, scheduling, revenue, incentives, alerts, and reports
Database Layer	Store operational, policy, schedule, revenue, and user data
Web Dashboard Layer	Provide a monitoring and decision-support interface for transport authorities
Mobile Application Layer	Provide schedules, alerts, trip summaries, earnings, incentives, and reporting for owners/drivers

Table 14 Component Interaction

3.2.2 Hardware and Software requirements with justification

The proposed system needs two components which include embedded hardware and embedded software systems. The hardware component operates to gather real-time bus operation data whereas the software component handles all processes including data processing and data storage and data analysis and data visualization and data reporting.

Hardware Requirements

Hardware Component	Requirement	Justification
ESP32 Microcontroller	Used as the main on-board processing unit	ESP32 is suitable because it has built-in Wi-Fi, low power consumption, enough processing capability for prototype-level event processing, and Arduino framework support.
NEO-6M GPS Module	Used to collect GPS coordinates, speed, and timestamp data	This module provides low-cost GPS data required for bus location tracking, route compliance checking, trip monitoring, and distance calculation.
MPU6050 IMU Sensor	Used to collect acceleration and gyroscope data	This sensor supports driver behavior monitoring such as harsh braking, rapid acceleration, sharp turning, and abnormal movement detection.
Vehicle Power Supply / Voltage Regulator	Used to power the IoT device from the bus electrical system	A stable power supply is required to keep the IoT device operating continuously during bus trips.
GPS Antenna and Sensor Connections	Used to improve GPS reception and sensor communication	Accurate GPS and stable sensor connectivity are required for reliable monitoring and calculation.
Desktop / Laptop Computer	Used by authority officers and administrators	Required to access the web dashboard through a browser.
Android / iOS Mobile Device	Used by bus owners and drivers	Required to access mobile app functions such as schedules, alerts, trip

		summaries, earnings, and breakdown reporting.
Wi-Fi / 4G Communication Interface	Used to transmit event data from the bus to the backend	Wireless communication is required because buses are moving vehicles and cannot depend on fixed wired networks.

Table 15 Hardware Requirements

Software Requirements

Software / Platform	Requirement	Justification
Arduino IDE	Firmware development for ESP32	Used to write, test, and upload firmware to the IoT device.
Azure IoT Hub or Equivalent IoT Service	IoT data ingestion	Used to receive and manage device-generated event data securely.
Spring Boot	Backend REST API and business logic	Suitable for building scalable APIs, processing IoT events, applying business rules, and connecting with the database.
PostgreSQL	Primary database	Used to store structured system data such as users, routes, schedules, policies, trips, alerts, and revenue records.
React JS	Web dashboard development	Used to create an interactive dashboard for transport authority officers and administrators.

React Native	Mobile application development	Allows development of a mobile application for bus owners and drivers using a single codebase.
Leaflet / Mapbox GL JS	Map visualization	Used to display approved routes, bus locations, halts, junctions, and deviation points.
Google Maps Roads API or Equivalent	Route verification and road-snapping	Can support route validation and map-based accuracy improvement.
Git / GitHub	Version control	Supports collaborative development and source code management.
Postman	API testing	Used to test backend API requests and responses during development.

Table 16 Software Requirements

3.2.3 Risk Mitigation Plan with alternative solution identification

The system may encounter risks that stem from technical issues, operational problems, hardware failures, communication breakdowns, data breaches, and security vulnerabilities. The risk mitigation plan identifies primary risks together with their potential effects and methods of reduction and alternative solutions.

Risk	**Impact**	**Mitigation Plan**	**Alternative Solution**
GPS signal loss or low accuracy	Route tracking, distance calculation, and deviation detection may become inaccurate	Use geofence tolerance, route snapping, timestamp validation, and event verification	Use external GPS antenna, higher accuracy GPS module, or combine GPS with route/halt event validation.
Network connectivity failure	IoT event data may not reach backend in real time	Store selected event data temporarily on device and synchronize when network is restored	Use 4G instead of Wi-Fi, add SMS fallback for critical alerts, or use batch upload
IoT device power failure	Bus data collection may stop during operation	Use stable voltage regulator and device health monitoring	Add backup battery or power-failure alert
Sensor noise from MPU6050	Unsafe driving detection may produce false alerts	Apply filtering, threshold tuning, and validation using test data	Use a higher quality IMU sensor or combine GPS speed data with IMU readings
Incorrect route/policy data	Schedule, compliance, and revenue decisions may be wrong	Validate route data, time slots, and policy rules before activation	Allow manual correction and policy version rollback

Backend server failure	Dashboard, mobile app, and processing may become unavailable	Use regular backups, monitoring, and restart procedures	Deploy on cloud with failover or backup server
Database growth due to IoT events	Storage and performance may reduce over time	Archive old raw events, summarize historical data, and use date filters	Use cloud storage or separate analytics database
Unauthorized system access	Sensitive operational and financial data may be exposed	Apply authentication, role-based access control, HTTPS, and secure password handling	Add multi-factor authentication for authority/admin users
Route deviation false positives	Users may lose trust in alerts	Use tolerance distance, duration threshold, and map-matching validation	Allow manual review before penalty confirmation
User resistance from bus owners/drivers	Adoption may become difficult	Provide training, simple mobile UI, transparent fee/incentive logic, and pilot testing	Start with voluntary pilot routes before full deployment
Hardware cost increase	Large-scale deployment may become expensive	Use low-cost IoT hardware and phased deployment	Lease devices instead of a full upfront device purchase
Cloud or API cost increase	Operating cost may increase as fleet size grows	Use event-driven data transmission and optimized API calls	Use local server deployment or lower-cost cloud services

Table 17 Risk Mitigation Plan

3.2.4 Cost-Benefit Analysis for the Proposed Solution

The proposed system requires costs for IoT hardware and software development, backend/cloud infrastructure, network communication, and system maintenance, user training, and field testing. The system delivers operational, financial, regulatory, and social advantages which make its costs acceptable.

DESCRIPTION	AMOUNT (LKR)	TOTAL (LKR)
SOFTWARE		
React.js	6 000	
Spring Boot	10 000	
React native	10 000	
C++	7 000	
PostgreSQL	9 000	
Visual Code	5 000	
Software Cost		47 000
HARDWARE		
Router	6 000	
2 pc	40 000	
IoT Device	18 000	
Total Hardware Cost		64 000
NETWORK		
Data transfer and API usage	8 000	
Hosting and server deployment	7 000	
Total Networking Cost		15 000
MAINTENCE		
Client training and technical support	6 000	
Warranty and support services	10 000	
Security software and monitoring	4 000	
System maintenance agreement	5 000	
Total Maintenance Cost		25 000
Total Project Cost		**151 000**

Table 18 Budget

Expected Benefits

Benefit Area	Expected Benefit
Transport Authority Benefit	Better route monitoring, stronger policy enforcement, improved revenue visibility, and evidence-based decision-making
Revenue Benefit	Distance-based permit fee calculation can reduce unfair fixed-fee limitations and support better government revenue
Off-Peak Service Benefit	Incentive calculation can encourage bus owners to operate during off-peak/night periods
Bus Owner Benefit	Owners can view trip summaries, schedules, fee details, incentives, earnings-related information, and driver performance
Driver Benefit	Drivers receive clearer schedule guidance, route alerts, unsafe driving alerts, and breakdown reporting support
Operational Benefit	Reduced manual monitoring, fewer phone-call-based updates, faster alerting, and better reporting
Data Benefit	Historical operational data can support future transport planning and policy decisions

Table 19 Expected Benefits

3.3 Performance Requirements

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System shall meet performance requirements related to IoT data collection, event transmission, backend processing, dashboard response, mobile application response, route compliance detection, scheduling, revenue calculation, and alert generation. The system must process important bus events with minimal delay because it operates as a transport monitoring system and an operational decision-making tool.

Requirement ID	Performance Requirement	Target / Expected Performance
PR-01	IoT device data capture	The IoT device shall capture GPS, timestamp, speed, acceleration, and gyroscope data during bus operation.
PR-02	Event transmission	Important events such as trip start, trip end, halt detection, route deviation, unsafe driving, and breakdown status shall be transmitted to the backend with minimum delay when network connectivity is available.
PR-03	Backend API response time	The backend shall respond to normal dashboard and mobile application API requests within 2–4 seconds under normal operating conditions.
PR-04	Dashboard update time	The authority dashboard shall display updated bus status, alerts, and route information within 5 seconds after receiving valid backend data.
PR-05	Time-slot classification	The system shall classify bus trips into correct time slots, such as peak, normal, off-peak, and night periods, with at least 95% expected accuracy.
PR-06	Schedule adjustment decision	The system shall generate schedule adjustment suggestions or gap warnings within 2 seconds after receiving valid disruption or availability data.
PR-07	Route deviation detection	The system shall detect route deviation events using GPS, route polyline, geofence, or map-matching logic with high accuracy and minimum false alerts.
PR-08	Revenue calculation	The system shall calculate distance-based permit fees and off-peak incentives accurately using validated distance, time-slot, route, and policy data.
PR-09	Alert generation	Route deviation, unsafe driving, missed trip, breakdown, and policy violation alerts shall be generated and displayed as soon as the event is processed by the backend.

PR-10	Mobile app performance	The mobile application shall load schedules, trip summaries, alerts, earnings, permit fee details, and incentive details within 3–5 seconds under stable network conditions.
PR-11	Concurrent bus support	The backend shall support event data from multiple buses operating at the same time during pilot testing and shall be scalable for future route expansion.
PR-12	Database performance	The database shall store and retrieve trip records, IoT events, schedules, policies, alerts, and revenue records efficiently using filtering, indexing, and date-range queries.
PR-13	Memory usage	The IoT device shall only store temporary event data and unsent packets. Large historical data shall be stored in the backend database, not on the device.
PR-14	System availability	The system shall remain available during normal operational hours with minimum downtime. Temporary network failures shall not permanently lose important event data if buffering is available.

Table 20 Performance Requirements

3.4 Design Constraints

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System design process faces restrictions from hardware requirements and software requirements and communication needs and data needs and security needs and operational requirements. The system needs to function correctly when buses operate because it serves as an embedded IoT-based transport management system that connects to backend services and dashboards and mobile applications.

The on-board IoT device shall be designed using low-cost and easily available hardware components such as ESP32, GPS module, IMU sensor, voltage regulator, and related circuit components. The device design should take into account all operational conditions because buses will face power interruptions and their power system will operate at different voltage levels and the system will experience all environmental conditions.

The IoT device will handle essential event data processing instead of keeping extensive historical data stored in its local system. The system will implement an event-driven method to transmit data which will replace the need for perpetual GPS data streaming whenever it is applicable. The design constraint serves to minimize expenses associated communication and to decrease bandwidth needs and to decrease cloud processing requirements and to limit database size expansion. The device will send essential operational information which includes the start of a trip and the end of a trip and the detection of stops and the deviation from the planned route and the occurrence of unsafe driving and the device's status of being offline.
The software design must adhere to the chosen technological framework which the project has selected. The backend system will use Spring Boot for its development work while PostgreSQL will serve as the database system. The web dashboard will be built using React while the mobile application will use React Native. The IoT firmware development will use Arduino IDE or any similar embedded development platform. The system allows for secure IoT communication through Azure IoT Hub and other equivalent IoT ingestion services. The project proposal identifies these technologies as components of the proposed system environment.

The system requires access to precise data regarding routes, schedules, halts, junctions, policies, permit fees, and incentives. The completeness and accuracy of official transport authority data will determine the effectiveness of route compliance monitoring and time-slot classification and schedule adjustment and revenue calculation processes. The system design will permit authorized users to change route information and policy rules and time slots and rate values without needing to access the system's main source code.

The system requires role-based access control as its fundamental design approach. Transport authority officers system administrators bus owners and drivers will be able to access only the system functions which correspond to their job responsibilities. The system will safeguard user credentials location records revenue information permit fee calculations driver behavior records and audit logs through authentication and authorization processes and secure communication methods and limited database access.
The system requires design that enables operation during network conditions which experience constant instability. The IoT device needs to store specific event data until the internet connection becomes available because buses operate in locations that have poor internet access. The system cannot provide complete updates to users while the network remains unavailable.

The dashboard and mobile application shall be designed for users with different levels of technical knowledge. Therefore, the user interface shall remain simple, readable, and role-based. Non-technical users cannot access advanced technical data but they will see essential alerts and schedules and route status and revenue summaries and reports.

The system requires modular design which enables users to update or enhance any single component of the system including the IoT device and event monitoring engine and policy and time-slot engine and revenue calculation module and dashboard and mobile application. The project needs this feature because it consists of four interdependent components which will require modifications after stakeholders provide feedback and the pilot testing phase.

The design approach must achieve a balance between six essential elements which include cost and reliability and scalability and security and usability and system maintainability. The system needs to operate as a testing prototype which can later expand to multiple buses and routes and all transport authority operational areas.

3.5 Software System Attributes

3.5.1 Reliability

The system needs to function correctly throughout standard bus operation periods while processing all authentic Internet of Things event data which it receives from its on-board equipment. The system needs to verify all GPS coordinates and timestamps and speed data, acceleration data and gyroscope readings and trip events and route deviations, schedule changes and revenue information before it allows storage or decision-making use of these records.

The system needs to decrease data loss threats by enabling temporary storage of specific Internet of Things events, which will occur during times when network access is not available. The system will transfer stored information to the backend system after network access restores. The backend system needs to process invalid data, duplicate data, delayed data, and incomplete Internet of Things data without causing system failures.

The system shall provide reliable outputs for:

- Route compliance monitoring
- Trip start and end detection
- Time-slot classification
- Schedule gap detection
- Distance-based revenue calculation
- Off-peak incentive calculation
- Unsafe driving event alerts
- Dashboard and mobile app reporting

3.5.2 Availability

The system will provide access to monitoring functions, schedule data, alert systems, and report content during public bus operational hours for transport authority officers, bus owners, and drivers. The backend system and database, dashboard interface, and mobile application will maintain functionality during typical network and server operational conditions.

The system shall be designed to continue partial operation during temporary network issues. The IoT device will store selected event data until it can send data, but will not send data immediately. Users of the mobile application can access recently loaded information during internet outages, but will need an internet connection for live updates.

The system shall support availability for the following major services:

Service	Availability Requirement
Backend APIs	Should be accessible for dashboard, mobile app, and IoT event processing
Database	Should be available for storing and retrieving route, trip, schedule, policy, alert, and revenue data
Authority Dashboard	Should be accessible for monitoring routes, schedules, alerts, and reports
Mobile Application	Should be accessible for owners and drivers to view schedules, alerts, earnings, and breakdown reporting
IoT Data Collection	Should operate during bus trips and synchronize data when network is available

Table 21 Availability Requirements

For future deployment, server monitoring, regular backups, restart procedures, and cloud-based hosting may be used to improve system availability.

3.5.3 Security

The system needs to safeguard user accounts and all operational data together with location records and driver behavior records and permit fee information and incentive records and revenue-related information. The system enables transport authorities and bus owners and drivers to use its functions but any unauthorized access or data manipulation will disrupt enforcement processes and payment systems and diminish public confidence.

The system requires its access control system to implement role-based access control. The system will permit system administrators and transport authority officers and bus owners and drivers to access only those functions and data which match their operational needs. The bus owner will only access information about their registered buses whereas the driver will receive access to trip details and alerts that their supervisor assigned to them.

The system shall include the following security controls:

Security Area	**Requirement**
Authentication	Users shall log in using valid credentials
Authorization	Access shall be controlled based on user role
Data Transmission	Web, mobile, and IoT communication should use secure communication such as HTTPS or MQTT over TLS
Password Protection	Passwords shall be encrypted or hashed before storage
Database Access	Direct database access from frontend or mobile app shall not be allowed
Audit Logs	Important actions such as policy changes, revenue updates, and manual overrides shall be logged
Data Privacy	Only required operational data shall be collected and stored

Table 22 Security

3.5.4 Maintainability

The system needs a modular design because this approach enables separate updates and testing and maintenance of its major components. The main modules include the IoT device firmware and event monitoring engine and policy and time-slot engine and revenue calculation module and driver monitoring module and backend APIs and database and web dashboard and mobile application.

The system needs to provide an update mechanism which enables users to change transport rules and operational values through configuration updates instead of making source code modifications. The system requires this functionality because time-slot definitions and route rules and grace periods and penalty thresholds and permit fee rates and incentive rates and bus schedules and route data will undergo changes throughout time.

The following functions/classes/modules are expected to change frequently:

Function / Module	**Reason for Change**
Time-Slot Rules	Peak, off-peak, and night-time definitions may change based on transport authority policy
Policy Rule Management	Grace periods, delay thresholds, penalties, and route-specific rules may be updated
Revenue Calculation Rules	Permit fee rates and incentive rates may change
Route Data Management	New routes, route changes, halts, and junctions may be added
Schedule Assignment Logic	Scheduling logic may be improved after pilot testing
Alert Thresholds	Speed, braking, turning, and route deviation thresholds may be adjusted
Dashboard Reports	Authority users may request new report formats
Mobile App Screens	Bus owner and driver feedback may require UI improvements

IoT Firmware	Sensor reading, buffering, and event detection logic may be refined

Table 23 Maintainability

3.6 Other Requirements

The IoT-Enabled Smart Bus Route Monitoring and Revenue Management System requires the implementation of main system components which include architecture, performance standards, design elements and software quality assessment methods together with additional system requirements that address ethical considerations and documentation needs and deployment processes and user experience design and data handling and system testing and ability to grow in the future.

The system shall follow ethical data handling practices because it collects GPS location data, driver behavior data, operational events, and revenue-related information.Only the minimum data required for monitoring scheduling route compliance revenue calculation and reporting shall be collected. Driver and bus operation data will support transport improvement and safety and compliance monitoring and operational decision-making but not for unnecessary surveillance purposes. The proposal includes ethical considerations which include informed participation and anonymous driver-vehicle identification and encrypted communication and restricted database access and minimum data collection.

The system shall provide proper documentation for development, deployment, operation, and maintenance. User manuals for transport authority officers and bus owners and drivers together with administrator guides for system configuration and API documentation for backend services and IoT device setup instructions and basic troubleshooting guidelines constitute the complete documentation package. Users need these documents because they enable system operation with minimum developer assistance.

The system requires both auditability and traceability for its operational functions. The system needs to create timestamps which show who performed these activities when users logged into the system and when policies were modified and when time slots and schedules and manual systems and route deviations and revenue systems and incentive systems and alert systems were updated. The system functions as a critical component which enables regulatory bodies to make decisions and execute financial assessments and resolve conflicts which arise between transport authorities and bus operators.

The system requires backup procedures which include data recovery mechanisms. The database needs to create backups of its records which include user information and bus details and route data and policy information and schedule data and trip records and revenue information and alert data and violation reports and audit records. The authorized administrators will have access to restore vital information through backup systems when a system breakdown occurs. The system uses long-term raw IoT event data for archiving or summarizing purposes which help to decrease storage requirements.

The system requires both simulated testing and actual testing through its operational scenarios before it becomes operational. The testing process will include testing of IoT device data collection and GPS precision and sensor output and route observation and route deviation identification and time-slot evaluation and schedule modification and revenue assessment and driver behavior notifications and dashboard operations and mobile application operations and security measures and system efficiency. The project proposal identifies multi-layer validation which includes hardware testing and algorithm verification and revenue assessment and system performance evaluation and user acceptance testing.

The system shall provide basic training for main users. Transport authority officers need training to operate the dashboard and track routes and develop policies and access reports and respond to alerts. Bus owners need training to operate the mobile application and access schedules and view permit fee and incentive information and check trip summaries. The training program will instruct drivers about their assigned trips and how to receive alerts and report equipment failures.

The system requires a design which enables its growth needs to be fulfilled in the future. The system will support additional buses and routes and users and policy rules and reports and transport regions through its future development although its initial prototype or pilot will test Route 138 and Western Province routes. The next software release will introduce SMS alerts and advanced analytics and forecasting and external government system integration and province-wide deployment support.

The system will enable non-technical users to operate it without any difficulties. The system will present its interface through basic design which produces clear user interface components and alert notifications and report content and mobile display content. The system should present user-facing screens in simple language which all users including transport authority officers and bus owners and drivers can understand.

The proposed embedded IoT-based transport management system must meet all these requirements both to function properly and to protect ethical standards and provide user satisfaction and system maintenance and auditing and security and pilot testing and future development.

4. Supporting Information

4.1APPENDICES

Appendix A – Survey / Interview Instrument

The appendix contains information collected through stakeholder discussions and consultations on public transport operations. The research data and insights were primarily derived from the Road Passenger Transport Authority, which included route tracking data, scheduling information, and details about bus operation difficulties shared by the system administrator.

Appendix B - Detailed System Diagrams

The appendix presents complete technical diagrams that were utilized throughout the system development process. The system consists of three main components, which include overall system architecture, component diagrams, and process flow diagrams to demonstrate how the IoT monitoring devices work with the cloud processing layer and Policy & Time-Slot Engine in the system.

Appendix C - Preliminary UI mockups and Codes

Sample UI for Web Application Dashboard & Mobile Application

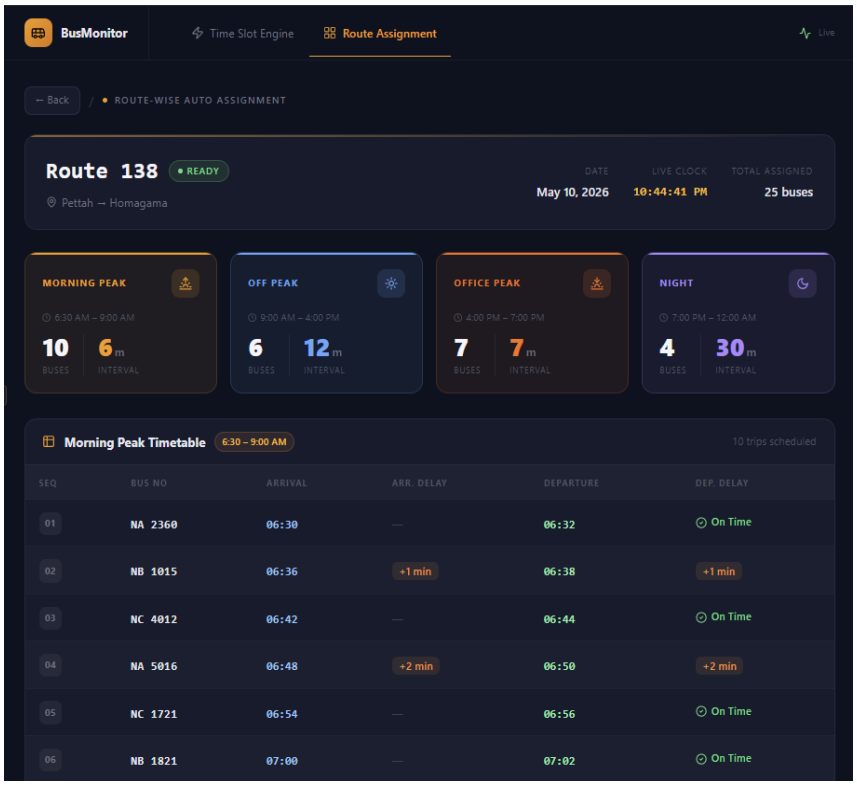


Figure 3 UI/ UX Route Assignment

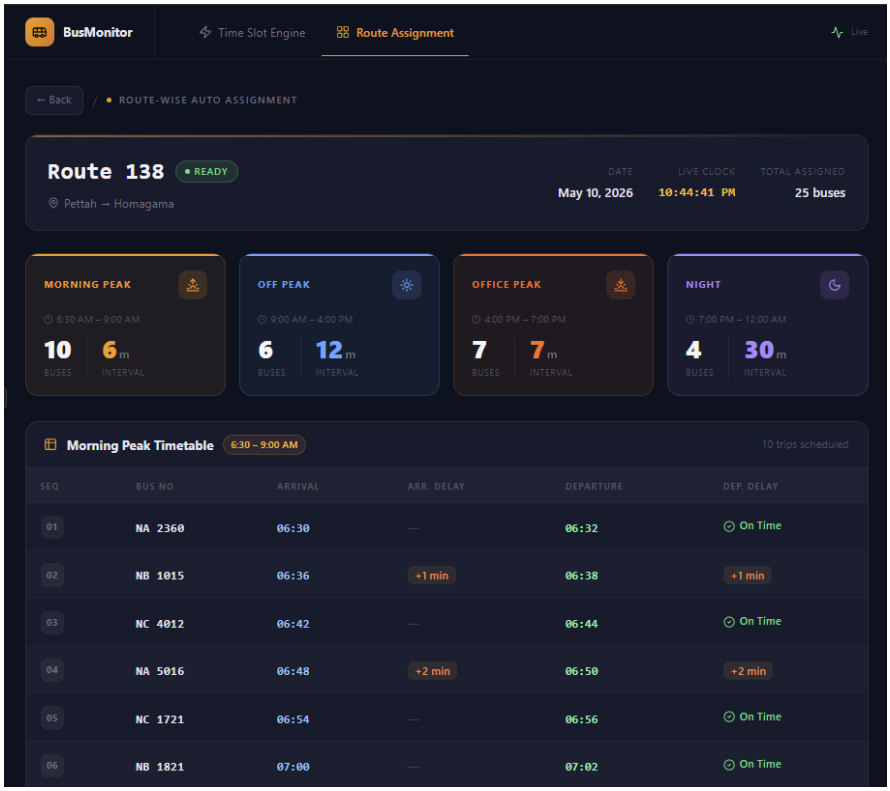


Figure 4 UI/ UX Route

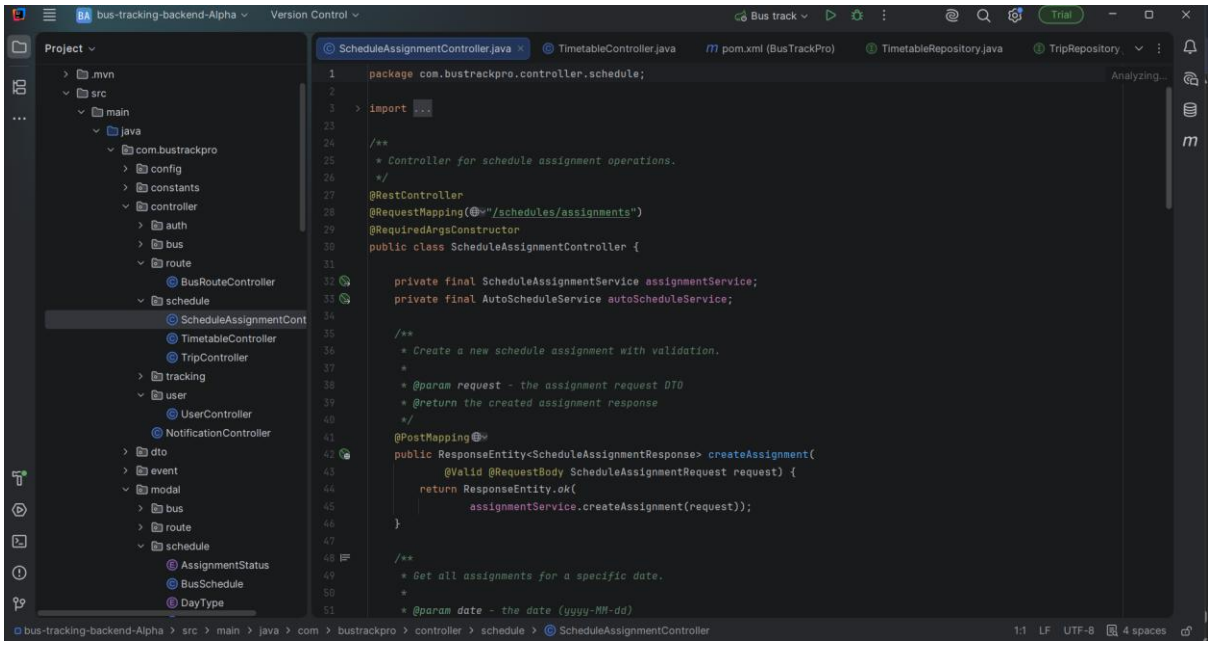


Figure 5 VS Code Sechedule

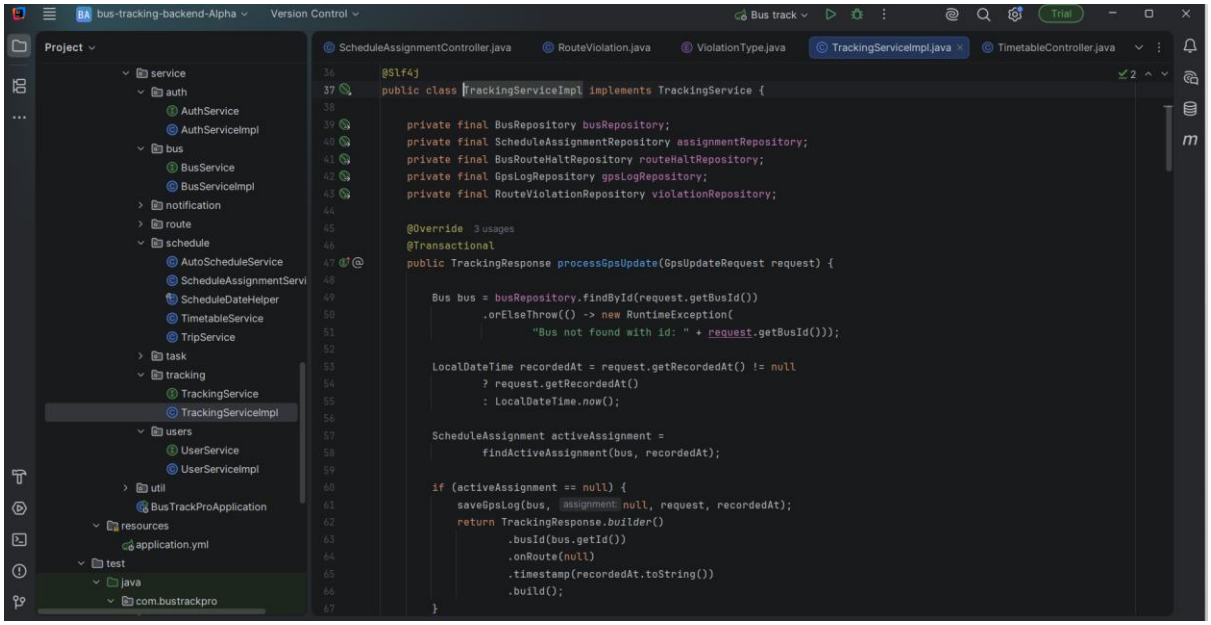


Figure 6 VS Code Tracking Backend

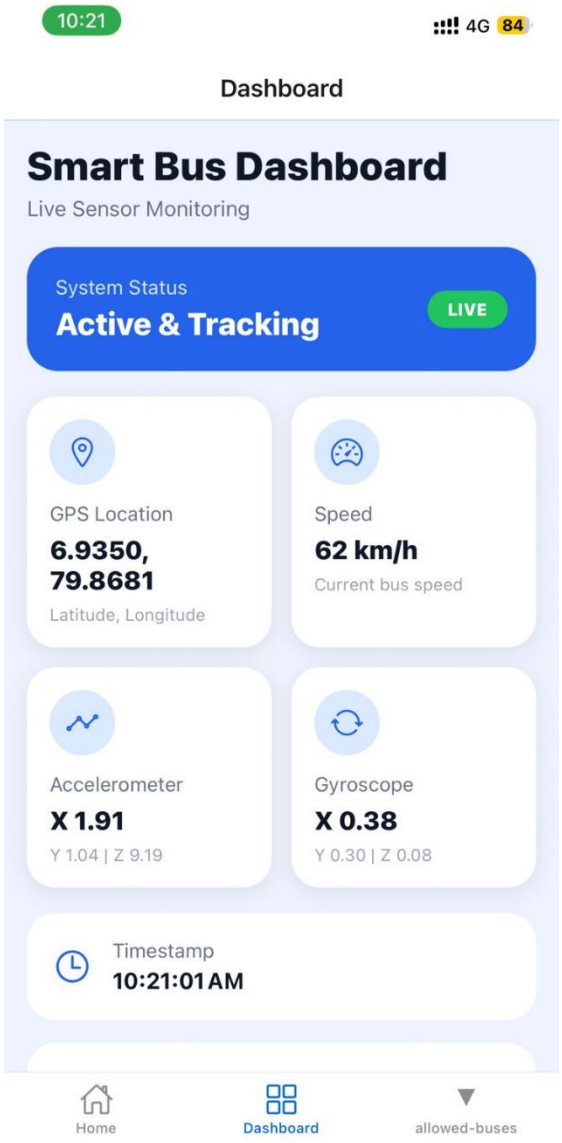


Figure 7 UI/ UX App 1

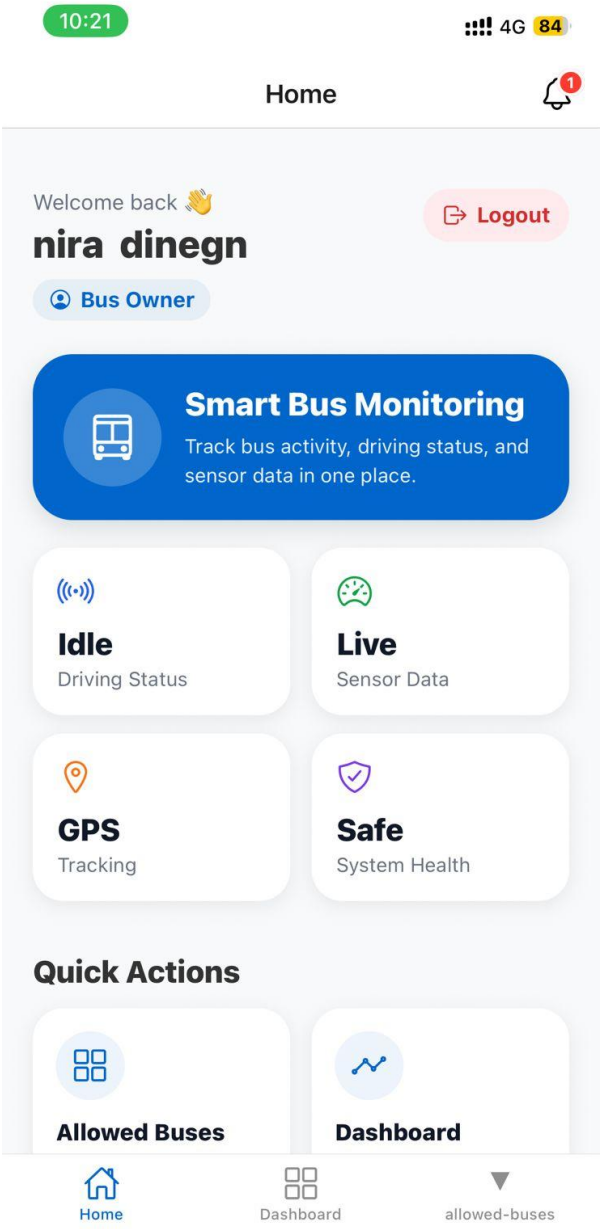


Figure 8 UI/ UX App 2

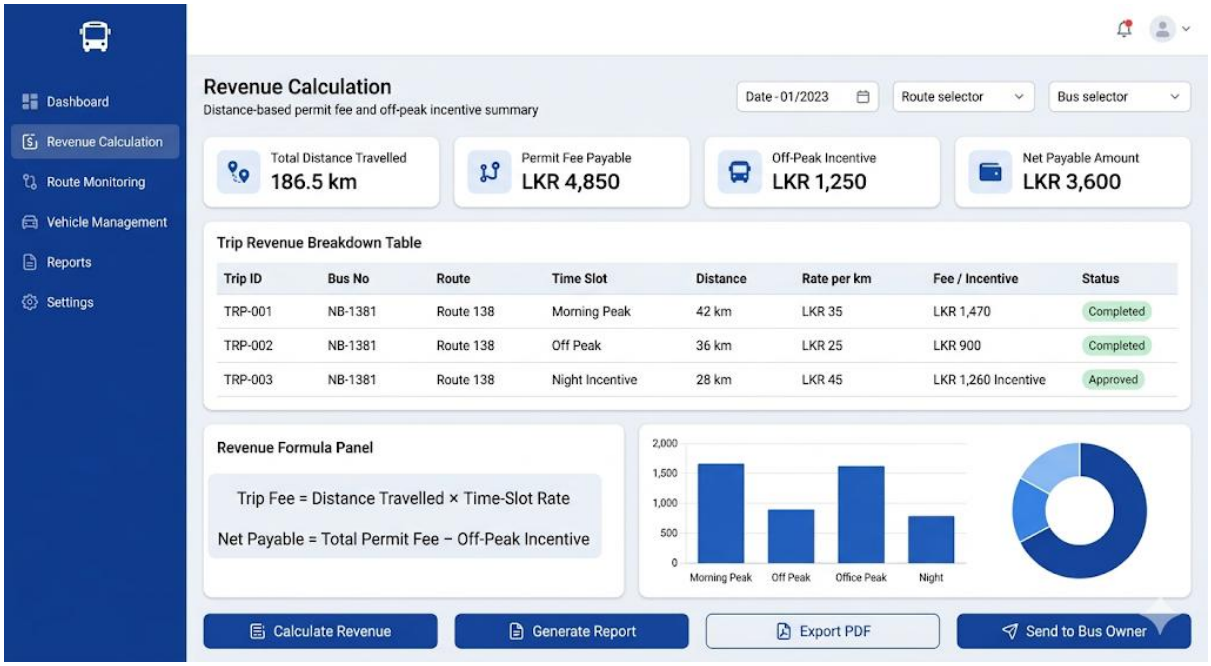


Figure 9 Revenue Calculatin

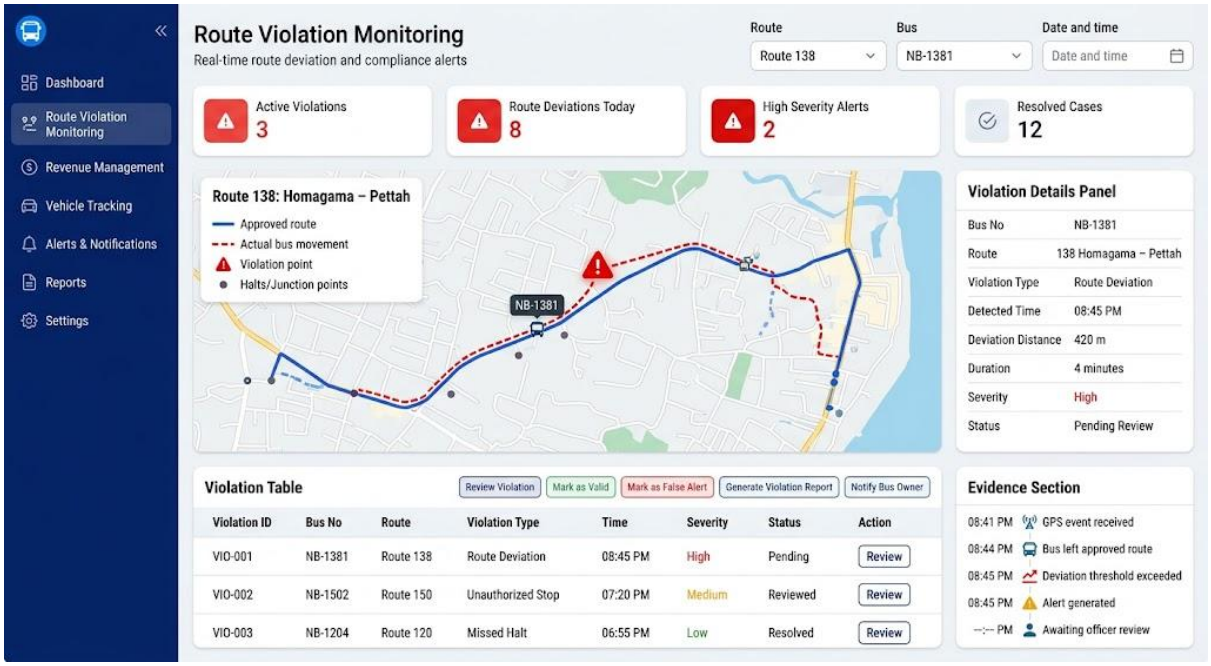


Figure 10 UI/ UX Tracking

Appendix D – Hardware Devices



Figure 11 Device Making step 1

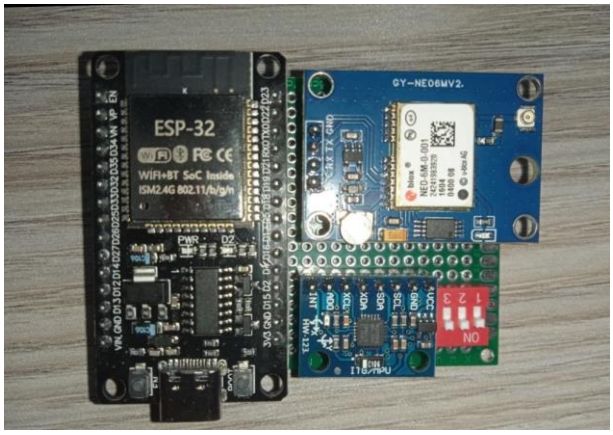


Figure 12 Device Making step 2

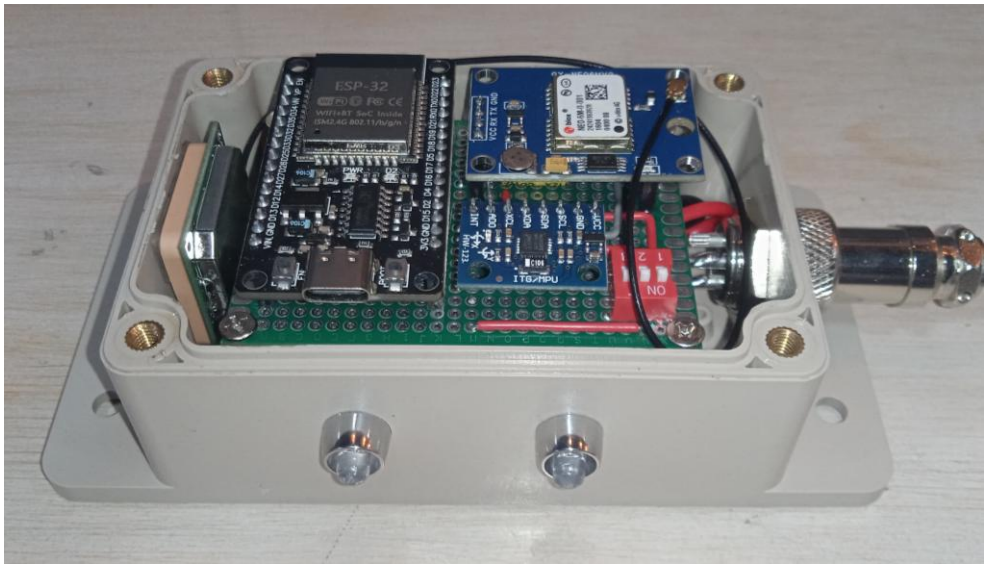
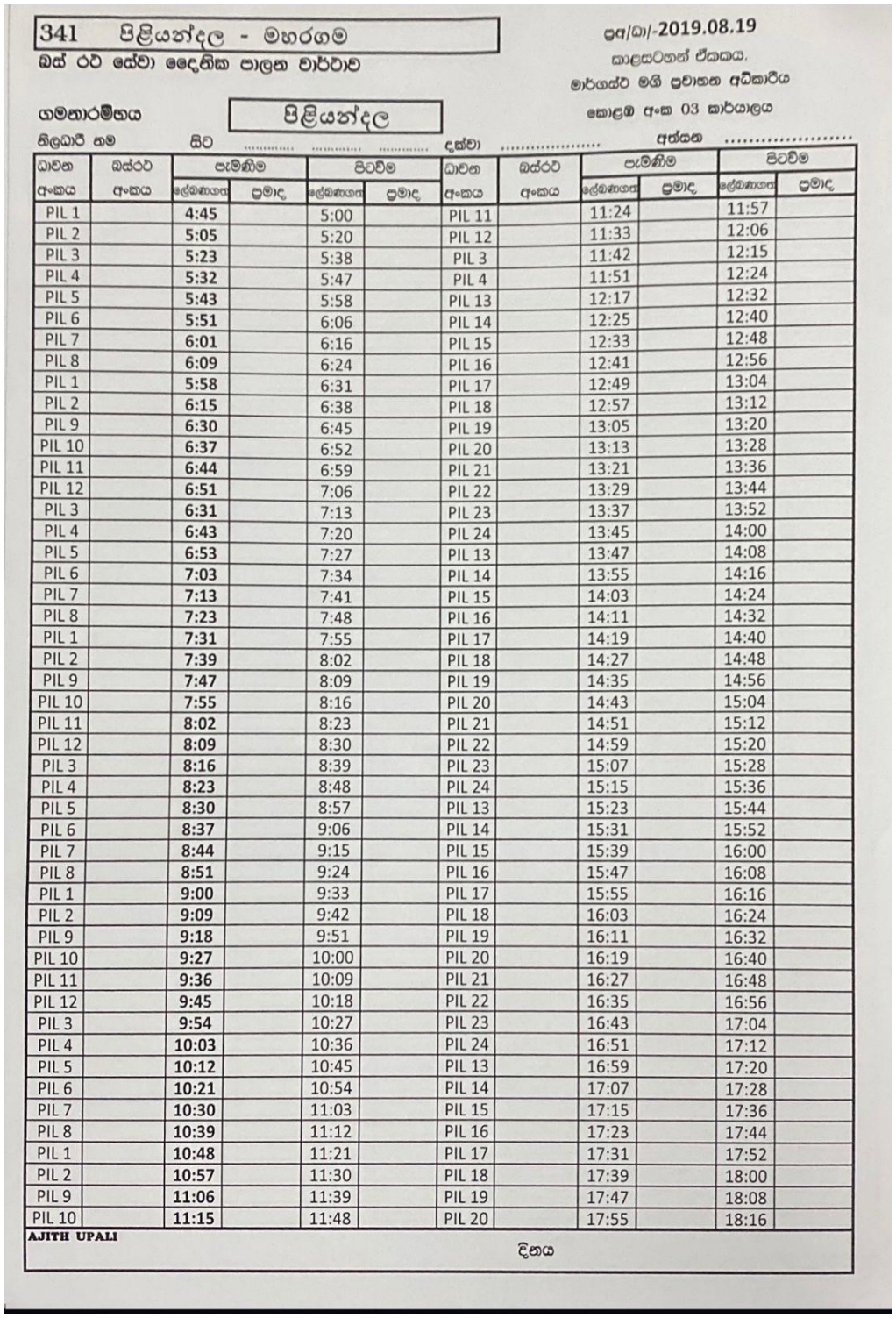


Figure 13 Final Device

Appendix E - Ethical documents

Current use bus schedule



341 පිළියන්දල - මහරගම

බස් රථ සේවා දෛනික පාලන වාර්තාව

පෙ/ධා/-2019.08.19

කාළසටහන් ඒකකය.

මාර්ගස්ථ මගී ප්‍රවාහන අධිකාරිය

කොළඹ අංක 03 කාර්යාලය

ගමනාරම්භය: පිළියන්දල

නිලධාරී නම ..... සිට ..... දක්වා ..... අත්සන .....

ධාවන අංකය	බස්රථ අංකය	පැමිණීම		පිටවීම		ධාවන අංකය	බස්රථ අංකය	පැමිණීම		පිටවීම	
		ලේඛණගත	ප්‍රමාද	ලේඛණගත	ප්‍රමාද			ලේඛණගත	ප්‍රමාද	ලේඛණගත	ප්‍රමාද
PIL 1		4:45		5:00		PIL 11		11:24		11:57	
PIL 2		5:05		5:20		PIL 12		11:33		12:06	
PIL 3		5:23		5:38		PIL 3		11:42		12:15	
PIL 4		5:32		5:47		PIL 4		11:51		12:24	
PIL 5		5:43		5:58		PIL 13		12:17		12:32	
PIL 6		5:51		6:06		PIL 14		12:25		12:40	
PIL 7		6:01		6:16		PIL 15		12:33		12:48	
PIL 8		6:09		6:24		PIL 16		12:41		12:56	
PIL 1		5:58		6:31		PIL 17		12:49		13:04	
PIL 2		6:15		6:38		PIL 18		12:57		13:12	
PIL 9		6:30		6:45		PIL 19		13:05		13:20	
PIL 10		6:37		6:52		PIL 20		13:13		13:28	
PIL 11		6:44		6:59		PIL 21		13:21		13:36	
PIL 12		6:51		7:06		PIL 22		13:29		13:44	
PIL 3		6:31		7:13		PIL 23		13:37		13:52	
PIL 4		6:43		7:20		PIL 24		13:45		14:00	
PIL 5		6:53		7:27		PIL 13		13:47		14:08	
PIL 6		7:03		7:34		PIL 14		13:55		14:16	
PIL 7		7:13		7:41		PIL 15		14:03		14:24	
PIL 8		7:23		7:48		PIL 16		14:11		14:32	
PIL 1		7:31		7:55		PIL 17		14:19		14:40	
PIL 2		7:39		8:02		PIL 18		14:27		14:48	
PIL 9		7:47		8:09		PIL 19		14:35		14:56	
PIL 10		7:55		8:16		PIL 20		14:43		15:04	
PIL 11		8:02		8:23		PIL 21		14:51		15:12	
PIL 12		8:09		8:30		PIL 22		14:59		15:20	
PIL 3		8:16		8:39		PIL 23		15:07		15:28	
PIL 4		8:23		8:48		PIL 24		15:15		15:36	
PIL 5		8:30		8:57		PIL 13		15:23		15:44	
PIL 6		8:37		9:06		PIL 14		15:31		15:52	
PIL 7		8:44		9:15		PIL 15		15:39		16:00	
PIL 8		8:51		9:24		PIL 16		15:47		16:08	
PIL 1		9:00		9:33		PIL 17		15:55		16:16	
PIL 2		9:09		9:42		PIL 18		16:03		16:24	
PIL 9		9:18		9:51		PIL 19		16:11		16:32	
PIL 10		9:27		10:00		PIL 20		16:19		16:40	
PIL 11		9:36		10:09		PIL 21		16:27		16:48	
PIL 12		9:45		10:18		PIL 22		16:35		16:56	
PIL 3		9:54		10:27		PIL 23		16:43		17:04	
PIL 4		10:03		10:36		PIL 24		16:51		17:12	
PIL 5		10:12		10:45		PIL 13		16:59		17:20	
PIL 6		10:21		10:54		PIL 14		17:07		17:28	
PIL 7		10:30		11:03		PIL 15		17:15		17:36	
PIL 8		10:39		11:12		PIL 16		17:23		17:44	
PIL 1		10:48		11:21		PIL 17		17:31		17:52	
PIL 2		10:57		11:30		PIL 18		17:39		18:00	
PIL 9		11:06		11:39		PIL 19		17:47		18:08	
PIL 10		11:15		11:48		PIL 20		17:55		18:16	

AJITH UPALI

දිනය

Figure 14 Current use bus schedule